

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

IMPLEMENTACIÓN Y MONITOREO AUTOMATIZADO DE HONEYPOTS SSH

AUTORES

López, Santiago

Morato, Elizabeth

DIRECTOR

Ing. Cortez, Alberto

Automatización en Ciberseguridad · Honeypots · Analítica de Logs
Inteligencia de Amenazas

Mendoza, República Argentina

Abril de 2026

Resumen

La presente investigación aborda el diseño, la implementación incremental y la validación funcional de un laboratorio automatizado de honeypots orientado a la captura, el procesamiento, el almacenamiento estructurado, la indexación analítica, la visualización interactiva y el alertado en tiempo real de eventos de intrusión en un entorno académico controlado. El trabajo parte de una problemática ampliamente documentada en la literatura especializada: en instituciones educativas de recursos moderados y en equipos técnicos de pequeña escala, los datos generados por sensores de seguridad —especialmente los honeypots— suelen permanecer dispersos en archivos de log aislados, sin una arquitectura integrada que permita transformarlos en evidencia organizada, trazable, contextualizada e interpretable. Esta situación no solo limita el valor operativo del sensor, sino que también empobrece el aprendizaje que los estudiantes pueden extraer de él, reduciéndolo a una observación fragmentaria de líneas de texto sin estructura analítica de soporte. Para resolver esta limitación de manera sistemática, la investigación diseña e implementa un pipeline de monitoreo continuo que articula nueve tecnologías y servicios de código abierto o de uso gratuito en una arquitectura cohesionada: Cowrie como honeypot de media interacción para servicios SSH, Docker como plataforma de contenedorización transversal que garantiza el aislamiento y la reproducibilidad del entorno, n8n como motor de automatización y orquestación de flujos con lógica SOAR-lite, PostgreSQL como capa de persistencia estructurada con esquema relacional optimizado para consultas analíticas, Elasticsearch como componente de indexación y búsqueda de alta velocidad, Kibana como entorno de visualización analítica con dashboards interactivos, la API pública de VirusTotal para el análisis automático de hashes SHA-256 de archivos descargados por los atacantes, la API pública de ip-api.com para la geolocalización automática de IPs atacantes con resolución de país, ciudad e ISP, y Telegram como canal de notificaciones en tiempo real mediante un bot automatizado. El proyecto se estructuró en nueve fases de implementación incremental, cada una de las cuales amplió la cobertura funcional del sistema sobre la base de la validación positiva de la fase anterior. Las primeras cuatro fases establecieron la infraestructura fundamental: despliegue del honeypot, configuración de la base de datos, desarrollo del workflow de automatización con parser NDJSON y control de timestamps, e integración del stack analítico. Las fases cinco y seis incorporaron las pruebas controladas de ataque: primero mediante conexiones SSH manuales con ejecución de comandos de reconocimiento, y luego mediante un experimento sistemático de cuarenta sesiones de ataque automatizado combinando Hydra para fuerza bruta de credenciales y sshpass para sesiones completas con comandos de reconocimiento y descarga de archivos maliciosos. La fase siete marcó la transición del sistema desde el laboratorio local hacia la exposición a internet real mediante el aprovisionamiento de un servidor VPS en DigitalOcean, región Amsterdam AMS3, con Ubuntu 24.04 LTS, obteniendo así una IP pública directa sin restricciones de router ni port forwarding. Las fases ocho y nueve introdujeron las mejoras de enriquecimiento: la integración de VirusTotal para el análisis automático de reputación de archivos descargados, y la integración de ip-api.com para la geolocalización en tiempo real de las IPs atacantes. Los

resultados obtenidos respaldan funcionalmente las hipótesis de la investigación dentro del alcance experimental definido, sin pretender una validación estadística formal. El experimento controlado de cuarenta sesiones produjo doscientos diecinueve eventos registrados en PostgreSQL en un período de aproximadamente dos minutos, con veintiséis sesiones únicas identificadas mediante el campo session de Cowrie, veintiséis alertas automáticas entregadas por Telegram sin ninguna intervención humana, y un tiempo medio de detección estimado en aproximadamente treinta segundos, derivado de la mitad del intervalo de activación del Schedule Trigger de n8n y no de una medición individual evento-alerta. La tasa de automatización del pipeline alcanzó el cien por ciento para todos los eventos dentro del alcance definido. La exposición a internet real mediante el VPS confirmó que el honeypot comenzó a recibir ataques de bots externos en menos de treinta minutos desde su activación, capturando en las primeras veinticuatro horas de operación más de cuatrocientos eventos provenientes de IPs de múltiples países, incluyendo infraestructura cloud de empresas como Alibaba Cloud LLC y Microsoft Azure. Las alertas de Telegram generadas durante este período incluyeron, además de los datos básicos del evento, la geolocalización de la IP atacante con país, ciudad e ISP, y el resultado de la consulta a VirusTotal para los hashes de archivos descargados. La investigación concluye que una arquitectura basada exclusivamente en herramientas open source y servicios de API pública gratuita puede constituir una solución técnicamente viable, operativamente validada, académicamente defendible y pedagógicamente significativa para laboratorios universitarios de ciberseguridad, a un costo de operación de seis dólares mensuales por el servidor VPS, con todos los demás componentes completamente gratuitos. Esta relación costo-valor contrasta radicalmente con las plataformas SOAR comerciales equivalentes, cuyo licenciamiento oscila entre cuarenta mil y cien mil dólares anuales. Palabras clave: honeypot · Cowrie · automatización · ciberseguridad · n8n · PostgreSQL · Elasticsearch · Kibana · Docker · SOAR-lite · analítica de logs · VirusTotal · geolocalización · inteligencia de amenazas · VPS · pipeline · MTTD · SecOps · open source · SSH · intrusión · monitoreo continuo.

Abstract

This research addresses the design, incremental implementation, and functional validation of an automated honeypot laboratory aimed at capturing, processing, structured storage, analytical indexing, interactive visualization, and real-time alerting of intrusion events in a controlled academic environment. The work originates in a widely documented problem in the specialized literature: in educational institutions with moderate resources and small-scale technical teams, data generated by security sensors —particularly honeypots— tend to remain scattered across isolated log files, without an integrated architecture capable of transforming them into organized, traceable, contextualized, and interpretable evidence. This situation not only limits the operational value of the sensor but also impoverishes the learning that students can extract from it, reducing it to a fragmented observation of unstructured text lines without analytical support. To address this limitation systematically, the research designs and implements a continuous monitoring pipeline that integrates nine open-source or free-to-use technologies and services into a cohesive architecture: Cowrie as a medium-interaction honeypot for SSH services, Docker as the transversal containerization platform ensuring isolation and reproducibility, n8n as the SOAR-lite automation and orchestration engine, PostgreSQL as the structured persistence layer with a relational schema optimized for analytical queries, Elasticsearch as the high-speed indexing and search component, Kibana as the visual analytics environment with interactive dashboards, the public VirusTotal API for automatic SHA-256 hash analysis of files downloaded by attackers, the public ip-api.com API for automatic geolocation of attacking IPs with country, city, and ISP resolution, and Telegram as the real-time notification channel through an automated bot. The project was structured in nine incremental implementation phases, each expanding the system's functional coverage based on the positive validation

of the previous phase. The results functionally support the research hypotheses within the defined experimental scope, without claiming formal statistical validation. The controlled experiment of forty attack sessions produced two hundred and nineteen events recorded in PostgreSQL within approximately two minutes, with twenty-six unique sessions identified, twenty-six automatic alerts delivered via Telegram without any human intervention, and an estimated Mean Time To Detect of approximately thirty seconds, derived from the n8n Schedule Trigger interval rather than from per-event alert timestamp measurement. The automation rate reached one hundred percent for all in-scope events. Real-internet exposure via the VPS confirmed that the honeypot began receiving external bot attacks within thirty minutes of activation, capturing over four hundred events from multiple countries in the first twenty-four hours, including cloud infrastructure from companies such as Alibaba Cloud LLC and Microsoft Azure. The research concludes that an architecture based exclusively on open-source tools and free public APIs can constitute a technically viable, operationally validated, academically defensible, and pedagogically meaningful solution for university cybersecurity laboratories, at an operational cost of six US dollars per month for the VPS server, with all other components completely free. This cost-value ratio contrasts sharply with equivalent commercial SOAR platforms, whose licensing ranges from forty thousand to one hundred thousand US dollars annually. Keywords: honeypot · Cowrie · automation · cybersecurity · n8n · PostgreSQL · Elasticsearch · Kibana · Docker · SOAR-lite · log analytics · VirusTotal · geolocation · threat intelligence · VPS · pipeline · MTTD · SecOps · open source · SSH · intrusion · continuous monitoring.

Índice General

Capítulo 1: Introducción

- 1.1 Contexto del problema
- 1.2 Planteamiento del problema
- 1.3 Pregunta de investigación
- 1.4 Justificación
 - 1.4.1 Justificación tecnológica
 - 1.4.2 Justificación académica
 - 1.4.3 Justificación económica y social
- 1.5 Objetivo general
- 1.6 Objetivos específicos
- 1.7 Alcance y limitaciones
 - 1.7.1 Alcance del proyecto
 - 1.7.2 Limitaciones del proyecto

Capítulo 2: Marco Teórico

- 2.1 Ciberseguridad contemporánea: del perímetro al monitoreo continuo
- 2.2 Honeypots: fundamentos, taxonomía y valor analítico
 - 2.2.1 Definición y propósito
 - 2.2.2 Taxonomía por nivel de interacción
 - 2.2.3 Valor analítico de los honeypots para la inteligencia de amenazas
- 2.3 Cowrie: arquitectura y funcionamiento en detalle
 - 2.3.1 Historia y contexto de desarrollo
 - 2.3.2 Estructura de los registros generados
- 2.4 El paradigma SOAR y su implementación académica
 - 2.4.1 Origen y evolución del paradigma SOAR
 - 2.4.2 n8n como implementación del paradigma SOAR-lite
- 2.5 Inteligencia de amenazas y APIs de enriquecimiento
 - 2.5.1 El concepto de Cyber Threat Intelligence
 - 2.5.2 VirusTotal: arquitectura y uso académico
 - 2.5.3 ip-api.com: geolocalización de IPs para contexto de amenazas
- 2.6 Infraestructura de análisis: PostgreSQL, Elasticsearch y Kibana
 - 2.6.1 PostgreSQL como capa de persistencia estructurada
 - 2.6.2 Elasticsearch como motor de indexación analítica
 - 2.6.3 Kibana como entorno de visualización analítica
- 2.7 Docker como tecnología de contenedorización transversal
- 2.8 Métricas de evaluación en sistemas de detección de intrusiones

Capítulo 3: Marco Metodológico

- 3.1 Naturaleza y tipo de investigación
- 3.2 Enfoque metodológico mixto
- 3.3 Diseño de investigación: laboratorio con validación funcional incremental
- 3.4 Hipótesis de investigación
 - 3.4.1 Hipótesis principal
 - 3.4.2 Hipótesis secundaria sobre reproducibilidad potencial
 - 3.4.3 Hipótesis secundaria sobre riqueza analítica
 - 3.4.4 Hipótesis secundaria sobre tráfico real
- 3.5 Variables e indicadores
 - 3.5.1 Variable independiente principal
 - 3.5.2 Variables dependientes
- 3.6 Técnicas e instrumentos de recolección de datos
- 3.7 Procedimiento de investigación
- 3.8 Consideraciones éticas, legales y de seguridad

Capítulo 4: Desarrollo de la Investigación

- 4.1 Arquitectura general del sistema
- 4.2 Fase 1 y 2: Diseño del entorno y despliegue de Cowrie
 - 4.2.1 Configuración del entorno local con Docker
 - 4.2.2 Despliegue de Cowrie en el entorno local
- 4.3 Fase 3: Diseño e implementación de la base de datos PostgreSQL
- 4.4 Fase 4: Desarrollo del workflow de automatización en n8n
 - 4.4.1 Estructura general del workflow
 - 4.4.2 El nodo Code y el parser NDJSON
 - 4.4.3 Integración con PostgreSQL y Elasticsearch
 - 4.4.4 El subsistema de alertas: Filtro Atacantes, ip-api, IF, VirusTotal y Telegram
- 4.5 Fase 5 y 5B: Integración del stack analítico y primeras pruebas de ataque
 - 4.5.1 Configuración de Elasticsearch y Kibana
 - 4.5.2 Primera campaña de pruebas con Hydra desde VM Ubuntu
- 4.6 Fase 6: Experimento controlado de cuarenta sesiones
 - 4.6.1 Diseño del experimento
 - 4.6.2 Ejecución y resultados
- 4.7 Fase 7: Aprovisionamiento del VPS y exposición a internet real
 - 4.7.1 Selección del proveedor y configuración del servidor
 - 4.7.2 Despliegue de Cowrie en el VPS y primera captura de tráfico real
 - 4.7.3 Configuración de la sincronización automática de logs
- 4.8 Fase 8: Integración de VirusTotal y geolocalización de IPs
 - 4.8.1 Proceso de integración de VirusTotal
 - 4.8.2 Proceso de integración de ip-api.com

Capítulo 5: Resultados y Análisis

- 5.1 Resultados del experimento controlado
 - 5.1.1 Corpus de eventos generados
 - 5.1.2 Métricas de desempeño del pipeline
 - 5.1.3 Análisis de las alertas de Telegram recibidas
- 5.2 Resultados de la exposición a internet real
 - 5.2.1 Caracterización del tráfico real capturado
 - 5.2.2 Funcionamiento del enriquecimiento con tráfico real
- 5.3 Análisis de trazabilidad y coherencia del sistema
- 5.4 Límites de inferencia de los resultados

Capítulo 6: Discusión

- 6.1 Contrastación de las hipótesis con los resultados
- 6.2 Diferenciación del proyecto frente a enfoques tecnológicos similares
- 6.3 Implicaciones pedagógicas del laboratorio desarrollado
- 6.4 Posicionamiento del sistema en el ecosistema de herramientas de seguridad

Capítulo 7: Conclusiones

- 7.1 Trabajos futuros

Referencias Bibliográficas

Anexos

- Anexo A** — Arquitectura del sistema
- Anexo B** — Esquema de base de datos PostgreSQL
- Anexo C** — Código del nodo Parser NDJSON + Timestamp
- Anexo D** — Script de sincronización VPS `sync_vps.ps1`
- Anexo E** — Script del experimento de 40 sesiones `experimento_40.sh`
- Anexo F** — Workflow n8n completo
- Anexo G** — Dashboards de Kibana
- Anexo H** — Contenedores Docker activos en entorno local
- Anexo I** — Honeypot VPS activo en servidor Amsterdam
- Anexo J** — Resultados del experimento en DBeaver
- Anexo K** — Formato de alertas de Telegram

Capítulo 1: Introducción

1.1 Contexto del problema

La expansión acelerada de la transformación digital durante la primera mitad de la década de 2020 ha reconfigurado profundamente la manera en que las organizaciones públicas, privadas y educativas conciben su infraestructura tecnológica, gestionan sus operaciones cotidianas y protegen su información. La proliferación de sistemas de autenticación remota, aplicaciones web accesibles desde internet, servicios en la nube de múltiples proveedores, plataformas colaborativas distribuidas y mecanismos automatizados de intercambio de datos ha dado lugar a un ecosistema tecnológico sin precedentes en términos de conectividad y eficiencia operativa. Sin embargo, ese mismo ecosistema incrementa de forma sostenida y a menudo inadvertida la denominada superficie de ataque: el conjunto de vectores, protocolos, servicios y puntos de acceso que un atacante potencial podría explotar para comprometer la seguridad de la organización. En este escenario, la ciberseguridad ha dejado de ser una preocupación restringida a áreas técnicas especializadas para convertirse en una dimensión transversal e ineludible del funcionamiento institucional en todos sus niveles. Proteger sistemas ya no supone solamente establecer barreras perimetrales —firewalls, antivirus, filtros de contenido— sino también desarrollar capacidades de observación activa, registro sistemático, correlación de eventos, interpretación de patrones y respuesta oportuna frente a la actividad hostil. Esta evolución conceptual se refleja en los marcos de referencia internacionales más influyentes en la materia: el NIST Cybersecurity Framework, la familia de normas ISO/IEC 27000 y las guías de gestión de incidentes del NIST SP 800-61, entre otros, coinciden en señalar la detección y la respuesta como funciones tan críticas como la protección preventiva.

La velocidad con que se producen y evolucionan los ataques contemporáneos hace insuficiente cualquier estrategia basada exclusivamente en la revisión manual de registros o en mecanismos estáticos de defensa. Los bots automatizados que recorren internet en busca de servicios SSH expuestos, los scripts de fuerza bruta que prueban miles de combinaciones de credenciales por minuto, los malware que se descargan y ejecutan en segundos después de una autenticación exitosa: todos estos fenómenos operan a una velocidad y escala que excede ampliamente la capacidad de respuesta humana no asistida por automatización. La consecuencia directa de esta asimetría temporal es que el tiempo medio de detección —la métrica que expresa cuánto tarda un sistema en identificar que está siendo atacado— se convierte en un factor determinante de la gravedad del impacto: cuanto más tiempo permanece un ataque sin detectar, mayores son las probabilidades de que cause daño significativo. Los honeypots emergen en este contexto como herramientas particularmente valiosas para la observación de amenazas, ya que ofrecen una propiedad analítica que muy pocas fuentes de datos de seguridad pueden igualar: toda interacción con un honeypot es, por definición, no autorizada. A diferencia de un servidor web de producción, donde el tráfico legítimo puede enmascarar intentos de intrusión, o de un firewall, donde el volumen de registros puede superar la capacidad de análisis manual, el honeypot opera como un sensor de señal pura: no existe tráfico legítimo que pueda introducir

ruido analítico. Cada conexión, cada intento de autenticación, cada comando ejecutado dentro del entorno emulado es evidencia de actividad hostil susceptible de registro, análisis e interpretación. No obstante, la mera existencia de un honeypot funcional no resuelve el problema de la observabilidad de amenazas. Un honeypot que genera logs sin que exista una arquitectura capaz de procesarlos, normalizarlos, almacenarlos de forma consultable y presentarlos en una interfaz analítica no es cualitativamente diferente, desde la perspectiva del analista, de no tener sensor alguno: los datos existen, pero permanecen inaccesibles en la práctica. Este es precisamente el vacío que la presente investigación se propone llenar: la brecha entre la captura del evento y su transformación en evidencia organizada, contextualizada, enriquecida y accionable. El contexto institucional en el que se desarrolla esta investigación es la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional, Facultad Regional Mendoza. La carrera forma técnicos universitarios en programación con competencias en desarrollo de software, bases de datos, redes y sistemas. La ciberseguridad representa una dimensión aplicada de particular relevancia para este perfil profesional: los técnicos en programación son quienes implementan los sistemas que deben protegerse, quienes configuran los servidores que quedan expuestos a internet, quienes administran las bases de datos que contienen información sensible. Dotarlos de herramientas conceptuales y prácticas para comprender cómo funcionan los ataques, cómo se detectan y cómo se automatiza la respuesta constituye una contribución formativa de alto valor.

1.2 Planteamiento del problema

El ecosistema de herramientas open source disponibles en el ámbito de la ciberseguridad es, en términos cuantitativos, excepcionalmente rico. Existen herramientas maduras y bien documentadas para cada etapa del ciclo de monitoreo: Cowrie, Honeyd, T-Pot y Dionaea para la captura de eventos mediante honeypots; Logstash, Fluentd y Filebeat para la ingesta y transformación de logs; Elasticsearch, OpenSearch y Wazuh para la indexación y búsqueda; Kibana, Grafana y OpenSearch Dashboards para la visualización; n8n, Apache Airflow y Node-RED para la automatización de flujos; PostgreSQL, MySQL y InfluxDB para la persistencia estructurada. Sin embargo, la disponibilidad de estas herramientas por separado no equivale a la existencia de una arquitectura integrada que las articule en un pipeline coherente y pedagógicamente significativo. En la práctica de los laboratorios académicos de ciberseguridad, la situación más frecuente es que cada herramienta se enseña y se utiliza de forma

aislada: se despliega Cowrie como ejercicio de honeypots, se muestra Kibana como ejercicio de visualización, se configura PostgreSQL como ejercicio de bases de datos. Los logs que genera Cowrie rara vez llegan a Kibana sin pasos manuales de exportación, transformación y carga. Las alertas, cuando existen, suelen ser el resultado de revisiones periódicas manuales del archivo de logs, no de un sistema de detección automatizado. Esta fragmentación no es solo una limitación técnica: es también una limitación pedagógica, porque impide que el estudiante comprenda el ciclo completo del monitoreo de amenazas como proceso integrado. El problema específico que esta investigación aborda puede enunciarse con precisión técnica: cuando un

honeypot como Cowrie registra un evento de intrusión —sea un intento de autenticación, un comando ejecutado en la shell simulada o la descarga de un archivo malicioso—, ese evento queda almacenado en un archivo de texto en formato NDJSON. Para que ese dato bruto se convierta en información útil para un analista o estudiante, es necesario que ocurran al menos cinco transformaciones sucesivas: primero, el evento debe ser leído y parseado desde el archivo de texto crudo; segundo, sus campos relevantes deben ser identificados y normalizados; tercero, el evento debe ser almacenado en una estructura de datos consultable y relacionada con otros eventos de la misma sesión; cuarto, debe ser indexado en un motor de búsqueda que permita consultas rápidas y agregaciones estadísticas; quinto, debe estar disponible en una interfaz visual que permita identificar patrones, tendencias y anomalías sin necesidad de escribir consultas SQL o comandos de línea de órdenes. Si cualquiera de estas cinco transformaciones requiere intervención manual, el tiempo de detección aumenta, la trazabilidad del proceso se debilita y la escalabilidad del sistema se ve comprometida. A estas cinco transformaciones básicas, una arquitectura de monitoreo moderna debe agregar dos dimensiones adicionales que el análisis de honeypots clásico no contempla: el enriquecimiento contextual del evento con información de inteligencia de amenazas externa, y la notificación proactiva al

operador cuando se detectan eventos de criticidad elevada. El enriquecimiento contextual transforma un dato técnico —un hash SHA-256— en una conclusión analítica —«este archivo es detectado como malware por cuarenta y siete de setenta y dos motores antivirus»—. La notificación proactiva transforma un registro pasivo en un sistema activo de alerta, capaz de informar al operador en tiempo real sin que este deba revisar periódicamente los dashboards. Estos problemas —fragmentación del pipeline, ausencia de automatización, falta de enriquecimiento contextual y carencia de alertas proactivas— son especialmente pronunciados en contextos académicos y de recursos moderados, donde tampoco existe presupuesto para adquirir plataformas comerciales integradas que resuelvan estos desafíos de forma plug-and-play. El problema de investigación queda así claramente delimitado: la inexistencia, en el contexto académico local, de una arquitectura open source integrada, documentada, reproducible y validada que articule captura, procesamiento, enriquecimiento, almacenamiento, indexación, visualización y alertado de eventos de honeypots en un pipeline automatizado de bajo costo.

1.3 Pregunta de investigación

A partir del problema descripto en la sección anterior, la investigación se organiza alrededor de una pregunta principal que concentra su intención central: ¿de qué manera el diseño e implementación de un laboratorio automatizado de honeypots, basado en herramientas open source articuladas en un pipeline de monitoreo continuo y desplegado en un entorno académico controlado, puede mejorar la captura, el procesamiento, el enriquecimiento contextual y el análisis de eventos de intrusión SSH, reduciendo teóricamente la latencia operativa respecto de una revisión manual periódica de registros y generando alertas automáticas enriquecidas con inteligencia de amenazas en tiempo real? Esta pregunta es deliberadamente comprehensiva

porque articula en una sola formulación las tres dimensiones sustantivas que el trabajo busca abordar: la dimensión técnica —¿es posible integrar estas herramientas en un pipeline

funcional?—, la dimensión operativa —¿el sistema automatizado reduce teóricamente la latencia operativa respecto de una revisión manual periódica?— y la dimensión de inteligencia —¿el sistema puede enriquecer automáticamente los eventos con contexto geográfico y de reputación de malware?—. Su respuesta afirmativa constituiría, al mismo tiempo, la validación del diseño propuesto y una contribución al conocimiento sobre arquitecturas de monitoreo open source en contextos académicos. Junto a la pregunta principal, la investigación se articula alrededor de cinco preguntas derivadas que enriquecen el análisis y permiten abordar aspectos específicos del problema. La primera pregunta derivada indaga si es posible construir una solución de monitoreo de honeypots reproducible con costos de operación marginales y sin dependencia de plataformas propietarias comerciales. La segunda pregunta derivada examina si la automatización de las etapas de procesamiento —parseo, normalización, almacenamiento, indexación y alertado— reduce efectivamente la carga manual de análisis y mejora el tiempo medio de detección de eventos críticos. La tercera pregunta derivada investiga si el pipeline puede enriquecerse con inteligencia de amenazas externa —análisis de hashes en VirusTotal, geolocalización de IPs mediante ip-api.com— de forma automática y en tiempo real, sin intervención del operador. La cuarta pregunta derivada explora si la exposición del honeypot a internet real mediante un servidor VPS genera tráfico malicioso autónomo de fuentes externas en un período de tiempo razonable, validando así el sistema con condiciones de operación reales y no solo simuladas. La quinta y última pregunta derivada considera si el laboratorio resultante puede convertirse en una herramienta didáctica valiosa para la enseñanza de conceptos de monitoreo, detección de intrusiones y operaciones de seguridad en el marco de la Tecnicatura Universitaria en Programación.

1.4 Justificación

1.4.1 Justificación tecnológica

La justificación tecnológica de esta tesis reposa sobre un argumento central: la mera existencia de datos no equivale a la existencia de información útil. Un honeypot puede generar cientos de eventos por hora, pero si esos eventos permanecen confinados en un archivo de texto sin procesamiento, normalización ni indexación, su potencial analítico es prácticamente nulo en términos operativos. La arquitectura propuesta por este proyecto busca resolver sistemáticamente esa brecha mediante la construcción de un pipeline que convierte el dato crudo del sensor en evidencia organizada, enriquecida y visualizable, pasando por siete transformaciones automatizadas que no requieren ninguna intervención manual una vez que el sistema está en operación. Desde una perspectiva tecnológica, el aspecto más significativo del proyecto es la integración de APIs de inteligencia de amenazas externa —VirusTotal e ip-api.com— como componentes nativos del pipeline de monitoreo. Esta decisión transforma cualitativamente el sistema: un honeypot básico sabe que un atacante descargó un archivo; este

sistema sabe, adicionalmente, si ese archivo es reconocido como malware por la comunidad global de antivirus, cuántos motores lo detectan y cuál es el nivel de consenso sobre su malignidad. Un honeypot básico registra la IP del atacante; este sistema sabe también desde qué país, ciudad e ISP proviene el ataque, lo que permite identificar patrones geográficos, infraestructura cloud maliciosa y actores recurrentes. Esta capacidad de enriquecimiento contextual automático representa el núcleo de la contribución tecnológica del proyecto. La elección de herramientas open source no es meramente una restricción presupuestaria sino una decisión tecnológica fundamentada.

Las herramientas seleccionadas —Cowrie, n8n, PostgreSQL, Elasticsearch, Kibana, Docker— son soluciones maduras, ampliamente documentadas, con comunidades activas de desarrollo y soporte, y con historiales probados de uso en contextos de producción real. La decisión de usar herramientas open source garantiza que el sistema pueda ser reproducido, auditado, modificado y extendido por cualquier equipo académico o técnico sin barreras de licenciamiento. Esta reproducibilidad no es solo un valor académico abstracto: es una condición operativa concreta que determina si el laboratorio puede ser adoptado por otras instituciones o grupos de investigación.

1.4.2 Justificación académica

La justificación académica de esta investigación se articula en torno a tres argumentos complementarios que se refuerzan mutuamente. El primero se refiere a la relevancia del objeto de estudio para la formación en ciberseguridad. El segundo se refiere a la pertinencia del enfoque metodológico adoptado. El tercero se refiere a la originalidad de la contribución en el contexto de la Tecnicatura Universitaria en Programación de la UTN Facultad Regional Mendoza. Respecto de la relevancia del objeto de estudio: la formación en ciberseguridad en las carreras de programación y sistemas enfrenta un desafío pedagógico estructural. Los conceptos centrales de la disciplina —ataque, sesión, intrusión, trazabilidad, correlación, indicador de compromiso, inteligencia de amenazas— son inherentemente dinámicos y relacionales: solo adquieren significado pleno cuando se los observa en acción, en la secuencia de un evento real o verosímil. Un libro de texto puede definir qué es un ataque de fuerza bruta SSH; pero solo la observación directa de cómo un bot automatizado prueba miles de combinaciones de credenciales en segundos, cómo el sistema registra cada intento, cómo el pipeline procesa automáticamente esos registros y cómo llega una alerta al celular del operador en tiempo real puede generar la comprensión profunda y encarnada que la práctica profesional requiere.

El laboratorio desarrollado en esta tesis proporciona exactamente ese tipo de experiencia. Respecto de la pertinencia del enfoque metodológico: la investigación adopta un diseño de laboratorio con validación funcional incremental que es especialmente adecuado para el tipo de objeto que estudia. No se trata de una investigación de campo sobre el comportamiento de atacantes reales —lo que plantearía problemas éticos y metodológicos complejos—, sino de una investigación aplicada sobre el diseño y la validación de una arquitectura técnica en un

entorno controlado. Este tipo de diseño es estándar en la ingeniería de software y los sistemas de información, y está bien establecido en la literatura académica de ciberseguridad como metodología apropiada para trabajos de este alcance. Respecto de la originalidad de la contribución: no existe, en el ámbito de la UTN Facultad Regional Mendoza, un antecedente documentado de un laboratorio de honeypots que integre simultáneamente automatización de flujos, persistencia relacional, indexación analítica, enriquecimiento con VirusTotal, geolocalización de IPs, alertas de Telegram y exposición a internet real mediante VPS. La tesis documenta el proceso completo de construcción de este sistema —incluyendo las fallas encontradas y las soluciones adoptadas— de manera que pueda ser reproducido por otros equipos académicos. Esta documentación procesual es, en sí misma, una contribución original al conocimiento disponible en el contexto institucional.

1.4.3 Justificación económica y social

La justificación económica y social de esta investigación es quizás la más directamente comprensible y la más inmediatamente relevante para el contexto argentino. La ciberseguridad ha dejado de ser un lujo de las grandes corporaciones para convertirse en una necesidad operativa de cualquier organización que procese datos, administre sistemas o preste servicios a través de internet. Sin embargo, el acceso a herramientas profesionales de monitoreo y detección de amenazas sigue siendo económicamente prohibitivo para la enorme mayoría de las organizaciones del país: municipios, cooperativas, pequeñas y medianas empresas, hospitales, clínicas, escuelas y universidades que necesitan proteger su información pero no pueden pagar licencias de decenas de miles de dólares anuales.

El sistema desarrollado en esta tesis demuestra que es posible construir una plataforma de inteligencia de amenazas operativamente comparable —en sus funciones centrales— con soluciones comerciales SOAR de primera línea, utilizando exclusivamente herramientas open source y servicios de API pública gratuita, con un costo de operación de seis dólares mensuales correspondientes al servidor VPS. Este resultado tiene implicaciones sociales concretas: si una tecnicatura universitaria puede construir y validar este tipo de sistema como trabajo final de carrera, entonces es razonablemente posible que municipios, cooperativas y organizaciones de la sociedad civil puedan adoptarlo para mejorar su postura de seguridad sin incurrir en costos prohibitivos. Desde esta perspectiva, la investigación no solo produce un artefacto técnico académicamente valioso, sino también un modelo de implementación replicable que puede tener impacto social positivo más allá del ámbito universitario. La democratización del acceso a capacidades de monitoreo de seguridad es, en el contexto argentino y latinoamericano, una contribución tanto técnica como social.

1.5 Objetivo general

El objetivo general de la presente investigación consiste en diseñar, implementar y validar funcionalmente un laboratorio automatizado de honeypots que permita capturar, procesar, enriquecer, almacenar, indexar y analizar eventos de intrusión SSH en un entorno académico

controlado, utilizando exclusivamente herramientas open source y servicios de API pública gratuita, articulados en una arquitectura de pipeline continuo con lógica SOAR-lite, de manera que el sistema sea reproducible, de bajo costo operativo, pedagógicamente significativo y capaz de generar alertas enriquecidas con inteligencia de amenazas en tiempo real sin intervención humana.

1.6 Objetivos específicos

Para alcanzar el objetivo general enunciado, la investigación se propone los siguientes nueve objetivos específicos, ordenados según la secuencia de implementación del proyecto:

1. Implementar un honeypot de media interacción basado en Cowrie, desplegado mediante Docker, capaz de registrar intentos de autenticación SSH con las credenciales utilizadas, comandos ejecutados dentro de la shell emulada, hashes SHA-256 de archivos descargados por los atacantes e información de red del origen de cada conexión, tanto en entorno local como en servidor VPS con IP pública real.
2. Diseñar un esquema de base de datos en PostgreSQL que preserve la trazabilidad completa de las sesiones de ataque, relacionando todos los eventos de una misma sesión mediante el campo identificador de sesión de Cowrie, y que permita consultas analíticas sobre credenciales, comandos, hashes y patrones de ataque.
3. Desarrollar un workflow de automatización en n8n que implemente las funciones centrales de un pipeline SOAR-lite: lectura periódica del archivo de logs NDJSON de Cowrie, parseo y normalización de eventos, control de timestamps para evitar el reprocesamiento de eventos ya procesados, enrutamiento de los eventos hacia PostgreSQL, Elasticsearch y el subsistema de alertas, y clasificación de eventos por nivel de criticidad.
4. Integrar Elasticsearch como motor de indexación y Kibana como entorno de visualización, configurando dashboards interactivos que permitan analizar en tiempo real la distribución de comandos ejecutados por los atacantes, las IPs de origen más activas, la actividad temporal de ataques y los indicadores de compromiso representados por hashes de archivos maliciosos.
5. Implementar la integración con la API pública de VirusTotal para el análisis automático de hashes SHA-256 de archivos descargados por los atacantes, incorporando el resultado del análisis —número de antivirus que detectan el archivo como malware sobre el total consultado— en las alertas de Telegram correspondientes a eventos de tipo descarga de archivo.
6. Implementar la integración con la API pública de ip-api.com para la geolocalización automática de las IPs atacantes en tiempo real, obteniendo país, ciudad, ISP y coordenadas geográficas, e incorporando esta información contextual en todas las alertas de Telegram generadas por el sistema.
7. Aprovisionar un servidor VPS en DigitalOcean con Ubuntu 24.04 LTS, desplegar en él el honeypot Cowrie con IP pública directa, configurar la sincronización automática de logs mediante SCP y clave SSH sin contraseña, y validar el sistema con tráfico real de internet.

8. Diseñar y ejecutar un experimento controlado de cuarenta sesiones de ataque automatizado utilizando Hydra para fuerza bruta de credenciales y sshpass para sesiones completas con comandos de reconocimiento y descarga de archivos, calculando las métricas de MTDD, tasa de automatización y número de alertas generadas.
9. Documentar el proceso completo de implementación incluyendo todas las fases, decisiones de diseño, errores encontrados y soluciones adoptadas, de manera que el laboratorio sea reproducible por otros equipos académicos con conocimientos equivalentes a los de un egresado de la Tecnicatura Universitaria en Programación.

1.7 Alcance y limitaciones

1.7.1 Alcance del proyecto

El alcance de la presente tesis se circunscribe al diseño, la implementación incremental y la validación funcional de un laboratorio de ciberseguridad defensiva para la captura y análisis de eventos de intrusión SSH en un entorno académico controlado, con extensión a un servidor VPS de IP pública para la captura de tráfico real. El sistema cubre el ciclo completo de monitoreo desde el sensor hasta la alerta, incluyendo todas las etapas intermedias de procesamiento, enriquecimiento, almacenamiento e indexación. Tanto el honeypot local como el VPS utilizan Cowrie como sensor; ambos se integran con el mismo pipeline de n8n mediante sincronización automática de logs. El alcance incluye la validación del sistema con dos tipos de tráfico: tráfico controlado generado por los propios investigadores mediante Hydra y sshpass desde una VM Ubuntu en VirtualBox, y tráfico real de internet capturado automáticamente por el VPS a partir de bots y atacantes externos que encontraron el honeypot de forma autónoma. Esta doble validación —controlada y real— aporta evidencia funcional relevante y permite observar el comportamiento del sistema bajo condiciones distintas, aunque no constituye por sí misma una validación estadística generalizable. El alcance también incluye la documentación técnica detallada del proceso de construcción del sistema, organizada en fases que corresponden a las etapas del desarrollo incremental. Esta documentación, que forma el Anexo Técnico del proyecto, está diseñada para ser suficientemente detallada como para permitir la reproducción del sistema por otro equipo académico con conocimientos equivalentes. La reproducibilidad es un valor metodológico central en esta investigación, y el nivel de detalle de la documentación técnica refleja ese compromiso.

1.7.2 Limitaciones del proyecto

El proyecto reconoce explícitamente las siguientes limitaciones, que deben tenerse en cuenta al interpretar los resultados y al evaluar las posibilidades de transferencia o generalización de las conclusiones:

La primera y más significativa limitación es la cobertura de protocolo: el sistema monitorea exclusivamente el protocolo SSH en el puerto 2222. No cubre ataques web sobre HTTP o HTTPS, intentos de intrusión sobre FTP, SMTP, RDP, SMB u otros protocolos de red frecuentemente atacados. Un sistema de monitoreo completo para una organización real

requeriría honeypots especializados para cada uno de estos protocolos, lo que está fuera del alcance de este trabajo académico.

La segunda limitación es la naturaleza de la detección: el sistema detecta eventos basándose en reglas fijas definidas en el workflow de n8n, específicamente los tipos de evento `cowrie.login.success` y `cowrie.session.file_download`. No implementa detección basada en machine learning, análisis de comportamiento anómalo ni correlación estadística de patrones a lo largo del tiempo. Un atacante sofisticado que operase con técnicas que no generan estos tipos de evento específicos podría no ser detectado y alertado, aunque sí registrado en PostgreSQL y Elasticsearch.

La tercera limitación es la dependencia de la PC local para el procesamiento: el pipeline de n8n, PostgreSQL, Elasticsearch y Kibana corre en la máquina local de los investigadores y requiere que esta esté encendida para procesar eventos. El honeypot en el VPS opera de forma continua e independiente, acumulando logs, pero el procesamiento automático de esos logs y la generación de alertas solo ocurren cuando la PC local está activa y el script de sincronización está corriendo. Esto introduce un período de latencia variable en la detección para los eventos ocurridos mientras la PC local estaba apagada, que se resuelve cuando la PC se enciende y el sistema procesa los eventos acumulados.

La cuarta limitación es el carácter estimado del MTDD: el tiempo medio de detección reportado en este trabajo —treinta segundos en promedio— es una estimación basada en el intervalo de activación del Schedule Trigger de n8n, no un valor calculado mediante la medición precisa del tiempo entre la ocurrencia de cada evento y la entrega de la alerta correspondiente. Una medición rigurosa del MTDD requeriría instrumentar el sistema para registrar el timestamp exacto de la alerta de Telegram y calcularlo automáticamente para cada evento, lo que no fue implementado en esta versión del proyecto. La quinta limitación es el carácter pasivo del sistema: el honeypot detecta, registra y alerta, pero no toma ninguna acción de respuesta activa. No bloquea IPs, no cierra conexiones, no envía señuelos a los atacantes ni ejecuta ningún tipo de contramedida. Esto es una decisión de diseño deliberada —la respuesta activa plantea consideraciones éticas y legales que exceden el alcance de un trabajo académico— pero implica que el sistema es un observatorio y no una plataforma de defensa activa. La sexta y última limitación se refiere a los límites de las APIs gratuitas: la API gratuita de VirusTotal permite 500 consultas diarias y 4 por minuto; la API gratuita de ip-api.com permite 45 consultas por minuto.

En condiciones de operación normal del proyecto, estos límites son ampliamente suficientes. Sin embargo, en escenarios de alta actividad —un ataque coordinado que genere decenas de descargas de archivos en minutos— podrían alcanzarse los límites de rate, lo que resultaría en errores en el nodo HTTP Request de VirusTotal que serían capturados por el manejo de errores de n8n sin interrumpir el flujo principal.

Capítulo 2: Marco Teórico

2.1 Ciberseguridad contemporánea: del perímetro al monitoreo continuo

La ciberseguridad como disciplina ha experimentado una transformación conceptual profunda en las últimas dos décadas que resulta esencial comprender para encuadrar correctamente la propuesta de este trabajo. Durante los años noventa y la primera mitad de la década de 2000, el paradigma dominante era el de la seguridad perimetral: la idea de que era posible y suficiente construir una barrera entre la red interna de la organización y el exterior —representado por internet— para proteger los activos de información. El firewall era el símbolo de esta concepción: una muralla digital que separaba lo seguro de lo inseguro, permitiendo el tráfico autorizado y bloqueando el resto. Este paradigma comenzó a mostrar sus limitaciones estructurales a medida que las organizaciones adoptaban modelos de trabajo más distribuidos, servicios web accesibles desde internet y relaciones con proveedores externos que requerían conectividad entre redes. La noción de un perímetro claro y controlable se volvió crecientemente anacrónica: los datos ya no estaban confinados a servidores internos, los empleados accedían a sistemas corporativos desde redes no controladas, y los proveedores externos tenían acceso legítimo a fragmentos de la infraestructura interna. En este contexto, la pregunta ya no era solo «¿cómo evito que entren?» sino también «¿cómo detecto que ya entraron y cuánto daño causaron?». Esta evolución dio lugar al paradigma del monitoreo continuo, que los marcos de referencia internacionales más influyentes han codificado de formas diversas. En particular, el NIST Cybersecurity Framework 2.0, publicado por el National Institute of Standards and Technology en 2024, organiza su núcleo alrededor de seis funciones: Gobernar, Identificar, Proteger, Detectar, Responder y Recuperar. La incorporación explícita de la función Gobernar en esta versión refuerza la necesidad de integrar la gestión del riesgo, la toma de decisiones institucionales y la responsabilidad organizacional con las capacidades técnicas de protección, detección, respuesta y recuperación. Dentro de este marco, la detección conserva un papel central, ya que reconoce que la protección preventiva por sí sola es insuficiente y que la capacidad de detectar amenazas de forma temprana constituye una función de seguridad autónoma e irreducible.

La norma ISO/IEC 27001 incorpora controles explícitos de monitoreo y registro de eventos como parte de los requisitos del Sistema de Gestión de Seguridad de la Información. El estándar NIST SP 800-137, titulado «Information Security Continuous Monitoring for Federal Information Systems and Organizations», establece el monitoreo continuo como práctica recomendada y define los componentes que deben integrar un programa de monitoreo maduro. El concepto de tiempo medio de detección adquiere su significado más preciso en este marco. Omar y Abed (2023) señalan que el MTDD es la métrica que expresa la velocidad de respuesta de la capacidad de detección de una organización: cuanto más bajo es el MTDD, más rápidamente puede iniciarse la respuesta al incidente, más limitado es el tiempo disponible para el atacante y menor es potencialmente el daño causado. En contraposición, un MTDD elevado —que puede medirse en horas o días en organizaciones sin sistemas de detección

automatizados— significa que el atacante ha tenido tiempo suficiente para moverse lateralmente, escalar privilegios, exfiltrar datos o instalar persistencia. La automatización del pipeline de monitoreo es, en este sentido, no solo una mejora de eficiencia operativa sino una contribución directa a la reducción del riesgo de seguridad.

2.2 Honeypots: fundamentos, taxonomía y valor analítico

2.2.1 Definición y propósito

El término honeypot designa un recurso de información —sistema, servicio, archivo, cuenta u otro activo— cuyo valor reside exclusivamente en el hecho de ser objeto de uso no autorizado o ilícito por parte de un atacante. Esta definición, formulada en términos seminales por Spitzner (2002) en su obra fundacional «Honeypots: Tracking Hackers», captura con precisión la paradoja constitutiva del honeypot: a diferencia de cualquier otro activo de seguridad, cuyo valor reside en su integridad y disponibilidad para usuarios legítimos, el honeypot solo cumple su función cuando es atacado. No tiene usuarios legítimos. No procesa transacciones reales. No almacena datos valiosos. Su único propósito es atraer, registrar y analizar la actividad de atacantes en un entorno donde esa actividad no puede causar daño real pero sí puede ser estudiada con detalle. Esta naturaleza paradójica del honeypot tiene consecuencias analíticas de enorme importancia. Mientras que otras fuentes de datos de seguridad —firewalls, proxies, sistemas IDS— generan registros que mezclan tráfico legítimo con actividad potencialmente maliciosa, creando un problema de señal-ruido que complica el análisis, el honeypot genera registros de señal pura: toda conexión es no autorizada, todo intento de autenticación es sospechoso, todo comando ejecutado en la shell emulada es potencialmente relevante desde la perspectiva de la inteligencia de amenazas. Esta propiedad convierte al honeypot en una fuente de datos excepcionalmente limpia y analíticamente valiosa para el estudio del comportamiento de los atacantes.

2.2.2 Taxonomía por nivel de interacción

La clasificación más difundida y metodológicamente relevante de los honeypots se basa en el nivel de interacción que ofrecen al atacante, es decir, en la riqueza y el realismo del entorno con que el sistema señuelo responde a las acciones del atacante. Esta taxonomía distingue tres categorías principales que presentan distintos compromisos entre riqueza de evidencia generada, riesgo operativo y complejidad de administración. Los honeypots de baja interacción emulan servicios de forma muy limitada, respondiendo a conexiones con comportamientos predefinidos y estáticos. Su principal virtud es la seguridad: al no exponer ningún sistema operativo real ni ninguna shell interactiva, las posibilidades de que un atacante los comprometa y los utilice como plataforma de ataque contra terceros son mínimas. Su principal limitación es la pobreza de la evidencia que producen: un honeypot de baja interacción puede registrar que alguien intentó conectarse a un puerto y qué contraseñas probó, pero no puede registrar qué habría hecho el atacante si hubiera ingresado exitosamente, porque no existe un entorno de post-autenticación que explorar.

Los honeypots de media interacción ofrecen una experiencia sustancialmente más rica para el atacante: presentan un servicio emulado que responde de forma verosímil a una secuencia más amplia de acciones, incluyendo la autenticación —que siempre resulta exitosa, independientemente de la contraseña utilizada— y la ejecución de comandos dentro de una shell simulada que imita el comportamiento de un sistema operativo real. Cowrie es el representante más destacado de esta categoría para el protocolo SSH. La media interacción permite capturar evidencia sobre el comportamiento post-autenticación del atacante —qué comandos ejecuta, qué archivos busca o descarga, qué herramientas instala— sin exponer un sistema operativo real que podría ser comprometido y utilizado para atacar a terceros. La shell emulada responde a los comandos con salidas verosímiles pero no ejecuta nada con efectos reales fuera del contenedor aislado. Los honeypots de alta interacción exponen sistemas operativos reales o máquinas virtuales completamente funcionales, lo que permite que los atacantes interactúen con un entorno genuinamente real y que la evidencia generada sea de la mayor profundidad y fidelidad posibles. Sin embargo, esta riqueza tiene un costo operativo considerable: un honeypot de alta interacción comprometido puede ser utilizado como plataforma de ataque contra terceros, lo que plantea responsabilidades legales y éticas serias para quien lo administra. Su despliegue requiere medidas de contención robustas —sandboxing, monitoreo de tráfico saliente, aislamiento de red— que incrementan significativamente la complejidad y el costo de operación. Por estas razones, los honeypots de alta interacción son apropiados para investigadores con experiencia avanzada y entornos de laboratorio especialmente preparados, no para el contexto de un trabajo académico de tecnicatura.

2.2.3 Valor analítico de los honeypots para la inteligencia de amenazas

Más allá de su función como sensor de detección, los honeypots constituyen una fuente de inteligencia de amenazas de primer orden cuando su evidencia se procesa y analiza correctamente. Zhang et al. (2022) señalan que los registros de honeypots permiten construir perfiles de comportamiento de atacantes, identificar herramientas de ataque comúnmente utilizadas, catalogar credenciales frecuentemente probadas en ataques de fuerza bruta y detectar nuevas técnicas de post-explotación a medida que emergen. Esta información, cuando se sistematiza y se comparte con la comunidad de seguridad, contribuye directamente a la construcción del conocimiento colectivo sobre el panorama de amenazas vigente. En el contexto específico de los honeypots SSH, los principales tipos de evidencia de inteligencia que puede extraerse incluyen: los diccionarios de credenciales utilizados por los bots de fuerza bruta —que revelan cuáles son los usuarios y contraseñas más frecuentemente probados en internet—, las técnicas de reconocimiento empleadas en la fase post-autenticación —qué comandos ejecutan los atacantes cuando logran ingresar—, los indicadores de compromiso representados por los hashes SHA-256 de los archivos maliciosos que los atacantes intentan descargar e instalar, las direcciones de origen de los ataques —que permiten identificar rangos de IP maliciosos, infraestructura de nube utilizada para ataques y países de origen frecuentes— y las versiones de software cliente SSH utilizadas por los atacantes —que en algunos casos permiten identificar herramientas específicas de ataque—.

2.3 Cowrie: arquitectura y funcionamiento en detalle

2.3.1 Historia y contexto de desarrollo

Cowrie es un honeypot SSH y Telnet de media interacción desarrollado originalmente por Michel Oosterhof como una bifurcación del proyecto Kippo, al que extendió y mejoró sustancialmente. El proyecto se ha mantenido activo desde su creación y cuenta con una comunidad de contribuidores que añade periódicamente mejoras en términos de realismo del entorno emulado, formatos de log y capacidades de integración. Su adopción en la comunidad de investigación de ciberseguridad es amplia: múltiples estudios académicos sobre el comportamiento de bots SSH, análisis de malware y caracterización de actores de amenaza han utilizado Cowrie como plataforma de recolección de datos. Desde el punto de vista técnico, Cowrie está implementado en Python y utiliza la biblioteca Twisted para el manejo de conexiones de red. Puede ejecutarse directamente en el sistema anfitrión, en un entorno virtual o, como se hace en este proyecto, dentro de un contenedor Docker. La imagen oficial `cowrie/cowrie` disponible en Docker Hub simplifica enormemente el despliegue, encapsulando todas las dependencias y configuraciones necesarias en un contenedor reproducible que puede iniciarse con un único comando.

2.3.2 Estructura de los registros generados

Los eventos generados por Cowrie se almacenan en un archivo de logs en formato NDJSON —Newline Delimited JSON, también conocido como JSON Lines—, donde cada línea es un objeto JSON independiente y válido que describe un evento específico. Esta estructura es especialmente favorable para el procesamiento automatizado: a diferencia de los formatos de log basados en texto plano con separadores variables, el NDJSON puede ser parseado de forma fiable y predecible por cualquier biblioteca estándar de manejo de JSON sin necesidad de expresiones regulares complejas. Cada objeto JSON en el archivo de logs incluye invariablemente un campo `timestamp` con la marca de tiempo del evento en formato ISO 8601, un campo `eventid` con el identificador del tipo de evento, un campo `session` con el identificador único de la sesión SSH a la que pertenece el evento, un campo `sensor` con el identificador único del sensor Cowrie que generó el evento, y un campo `uuid` con un identificador único universal para el evento individual. Adicionalmente, según el tipo de evento, el objeto puede incluir campos específicos como `src_ip` y `src_port` para los eventos de conexión, `username` y `password` para los eventos de autenticación, `input` para los eventos de comando ejecutado, `url` y `shasum` para los eventos de descarga de archivo, y `duration` para los eventos de cierre de sesión. La presencia del campo `session` en todos los eventos es especialmente valiosa desde el punto de vista analítico: permite agrupar todos los eventos que corresponden a una misma conexión SSH —desde el evento de conexión inicial hasta el evento de cierre de sesión— y reconstruir la secuencia completa de acciones de un atacante durante esa sesión. Este campo es el que permite responder preguntas como: «¿Qué comandos ejecutó exactamente el atacante que ingresó con las credenciales `root/admin` a las 10:53 del día de hoy?» o «¿Qué archivos intentó descargar durante esa sesión y cuáles son sus hashes?».

2.4 El paradigma SOAR y su implementación académica

2.4.1 Origen y evolución del paradigma SOAR

Security Orchestration, Automation and Response —cuyo acrónimo, SOAR, se ha convertido en uno de los términos más influyentes en el vocabulario de las operaciones de seguridad durante la última década— designa simultáneamente un paradigma conceptual, una categoría de productos de software y un conjunto de prácticas operativas que buscan dar respuesta al problema de la escala en las operaciones de seguridad. El término fue acuñado por Gartner alrededor de 2015 para describir plataformas que integran tres capacidades previamente dispersas: la orquestación de múltiples herramientas de seguridad, la automatización de tareas repetitivas y la gestión de flujos de respuesta a incidentes. El problema al que SOAR responde es concreto y bien documentado: los centros de operaciones de seguridad modernos procesan volúmenes de alertas que superan ampliamente la capacidad de análisis humano. Estudios de la industria han reportado que un SOC de tamaño mediano puede recibir entre mil y diez mil alertas por día, de las cuales una fracción significativa son falsos positivos que consumen tiempo de analistas sin producir valor. En este contexto, la automatización de las tareas rutinarias de clasificación, enriquecimiento y enrutamiento de alertas libera la capacidad analítica humana para concentrarse en los casos genuinamente complejos que requieren juicio experto. En el contexto de esta investigación, se utiliza la expresión SOAR-lite para referirse a una implementación simplificada del paradigma SOAR, orientada a recuperar sus funciones centrales —orquestación, automatización, enriquecimiento y alertado— sin incorporar la complejidad, el costo ni la amplitud funcional de una plataforma corporativa completa. Esta categoría no se propone como una clasificación comercial formal, sino como una denominación operacional adoptada para describir el alcance técnico del sistema desarrollado. Bajo esta perspectiva, herramientas de automatización de flujos de trabajo de código abierto como n8n, Node-RED o Apache Airflow pueden cumplir funciones esenciales de orquestación y automatización en laboratorios académicos y organizaciones de recursos moderados. En este contexto, herramientas de automatización de flujos de trabajo de código abierto como n8n, Node-RED o Apache Airflow pueden cumplir las funciones esenciales de orquestación y automatización con una fracción del costo de las plataformas SOAR comerciales.

2.4.2 n8n como implementación del paradigma SOAR-lite

n8n es una plataforma de automatización de flujos de trabajo de código abierto que representa, en el contexto de este proyecto, el motor central del enfoque SOAR-lite. Desarrollada por n8n GmbH con licencia Fair-code, puede ser desplegada en modo self-hosted mediante Docker, lo que garantiza la privacidad de los datos y la ausencia de dependencias de servicios cloud de terceros para el funcionamiento del pipeline. Su arquitectura basada en nodos visuales permite construir flujos de trabajo complejos de forma declarativa, conectando bloques funcionales que representan acciones específicas —lectura de archivos, ejecución de código JavaScript, consultas HTTP, inserción en bases de datos, envío de mensajes— sin necesidad de escribir código de infraestructura para cada integración. En el contexto de esta investigación, n8n

implementa las tres funciones centrales del paradigma SOAR de la siguiente manera. La orquestación se concreta en la integración de seis componentes heterogéneos —el archivo de logs de Cowrie, PostgreSQL, Elasticsearch, la API de VirusTotal, la API de ip-api.com y el bot de Telegram— en un flujo único y coherente, eliminando la necesidad de scripts ad hoc separados para cada integración. La automatización se materializa en el Schedule Trigger que activa el pipeline cada sesenta segundos sin ninguna intervención del operador, en el parser NDJSON que transforma el texto crudo del log en objetos estructurados procesables, y en el control de timestamps que garantiza que cada evento sea procesado exactamente una vez. La respuesta se expresa en las alertas enriquecidas que llegan al celular del operador en tiempo real, incluyendo no solo los datos básicos del evento sino también la geolocalización del atacante y el resultado del análisis de VirusTotal.

2.5 Inteligencia de amenazas y APIs de enriquecimiento

2.5.1 El concepto de Cyber Threat Intelligence

La inteligencia de amenazas cibernéticas —Cyber Threat Intelligence o CTI en inglés— designa el conocimiento basado en evidencia sobre amenazas cibernéticas existentes o emergentes, incluyendo el contexto, los mecanismos, los indicadores, las implicaciones y los consejos accionables derivados de ese conocimiento, que puede ser utilizado para apoyar la toma de decisiones de seguridad. Este concepto, ampliamente desarrollado en la literatura académica y profesional de la última década, distingue cuatro niveles de madurez: la inteligencia estratégica, orientada a decisores de alto nivel; la inteligencia táctica, centrada en las técnicas, tácticas y procedimientos de los actores de amenaza; la inteligencia operacional, enfocada en campañas de ataque específicas; y la inteligencia técnica, que incluye indicadores de compromiso concretos como hashes de malware, IPs maliciosas y dominios de comando y control. Este proyecto opera principalmente en el nivel de la inteligencia técnica: la integración con VirusTotal permite obtener información de reputación sobre hashes específicos de archivos —¿este binario es conocido como malware por la comunidad de antivirus?—, y la integración con ip-api.com permite contextualizar las IPs atacantes con información geográfica y de proveedor. Aunque este nivel de inteligencia es el más básico de los cuatro, es también el más directamente accionable en el contexto de un sistema de monitoreo de honeypots: permite pasar de «un atacante descargó un archivo» a «un atacante desde Alibaba Cloud en Estados Unidos descargó un archivo detectado como malware por cuarenta y siete de setenta y dos antivirus».

2.5.2 VirusTotal: arquitectura y uso académico

VirusTotal es una plataforma de análisis de archivos, URLs, dominios e IPs que fue adquirida por Google en 2012 y que desde entonces opera como un servicio de acceso público que agrega los resultados de análisis de más de setenta motores antivirus y herramientas de seguridad. Su valor como servicio de referencia para la comunidad de seguridad reside en la amplitud de la cobertura —la probabilidad de que un malware conocido sea detectado por al menos uno de los setenta motores es alta incluso cuando los motores individuales aún no han actualizado sus

definiciones— y en la construcción de una memoria colectiva de reputación de archivos que se nutre de millones de submissions diarias de usuarios de todo el mundo. La API pública gratuita de VirusTotal —a la que se accede mediante una clave de API obtenida con el registro gratuito en la plataforma— permite consultar si un hash SHA-256 ya ha sido analizado por VirusTotal y cuáles son los resultados de ese análisis. La respuesta de la API incluye, entre otros campos, el objeto `last_analysis_stats` que reporta cuántos motores clasificaron el archivo como malicioso, cuántos como limpio, cuántos como sospechoso y cuántos como no detectado. En el contexto de este proyecto, `n8n` extrae el valor `malicious` de este objeto y el total de motores consultados, y construye con ellos la leyenda «Detectado por X de Y antivirus» que aparece en la alerta de Telegram correspondiente a cada evento de descarga de archivo.

2.5.3 ip-api.com: geolocalización de IPs para contexto de amenazas

La geolocalización de IPs es el proceso de inferir la ubicación geográfica aproximada de un dispositivo de red a partir de su dirección IP pública, utilizando bases de datos que correlacionan rangos de IPs con ubicaciones geográficas registradas en los organismos de registro de internet regionales (RIR). El servicio `ip-api.com` proporciona una API REST gratuita —para uso no comercial— que recibe una dirección IP como parámetro y devuelve un objeto JSON con información de país, código de país, región, nombre de región, ciudad, código postal, latitud, longitud, zona horaria, ISP, organización y número de sistema autónomo (AS). En el contexto de este proyecto, la integración con `ip-api.com` sirve múltiples propósitos. Primero, contextualiza geográficamente el origen de cada ataque, permitiendo identificar patrones como la concentración de ataques en determinados países o el uso de infraestructura cloud de proveedores específicos. Segundo, enriquece las alertas de Telegram con información comprensible para el operador humano: saber que un ataque proviene de «Alibaba Cloud LLC, Estados Unidos» es inmediatamente más significativo que conocer solo la dirección IP 47.251.178.17. Tercero, los campos de latitud y longitud proporcionados por la API están disponibles para una futura visualización en un mapa geográfico de Kibana, que permitiría mostrar el origen global de los ataques de forma visual e impactante.

2.6 Infraestructura de análisis: PostgreSQL, Elasticsearch y Kibana

2.6.1 PostgreSQL como capa de persistencia estructurada

PostgreSQL es un sistema de gestión de bases de datos objeto-relacional de código abierto, mantenido por una comunidad internacional de desarrollo conocida como PostgreSQL Global Development Group, que se ha posicionado como una de las plataformas más robustas y ricas en funcionalidades del ecosistema de bases de datos. Su historia se remonta a 1986, cuando el proyecto POSTGRES fue iniciado en la Universidad de California en Berkeley bajo la dirección de Michael Stonebraker. La versión actual soporta un conjunto extenso de tipos de datos —incluyendo JSON nativo, arrays, tipos geométricos y rangos—, transacciones ACID completas, consultas complejas con múltiples joins y subconsultas, índices de múltiples tipos, procedimientos almacenados en múltiples lenguajes y replicación tanto física como lógica. En

el contexto de este proyecto, PostgreSQL cumple la función de repositorio estructurado y persistente de los eventos capturados por Cowrie. La elección de un sistema de base de datos relacional frente a alternativas como bases de datos NoSQL documentales o bases de datos de series temporales responde a varias consideraciones. Primero, la naturaleza relacional de los datos de Cowrie: los eventos de una sesión están relacionados entre sí mediante el campo `session`, y esta relación es más expresable y consultable en un modelo relacional que en uno documental. Segundo, la familiaridad de los estudiantes de la Tecnicatura con SQL como lenguaje de consulta, lo que facilita el uso pedagógico del sistema. Tercero, la madurez y estabilidad de PostgreSQL como plataforma, que garantiza la integridad de los datos incluso en condiciones adversas.

2.6.2 Elasticsearch como motor de indexación analítica

Elasticsearch es un motor de búsqueda y análisis distribuido de código abierto, desarrollado sobre Apache Lucene y mantenido por Elastic N.V. Su modelo de datos es orientado a documentos: cada evento es almacenado como un documento JSON en un índice, y el motor construye automáticamente estructuras de índice invertido que permiten búsquedas de texto completo, consultas de términos exactos, consultas de rango y agregaciones estadísticas con tiempos de respuesta de milisegundos incluso sobre colecciones de millones de documentos. En el contexto de este proyecto, Elasticsearch cumple la función de capa de indexación que alimenta los dashboards de Kibana. Mientras que PostgreSQL está optimizado para consultas relacionales precisas —«dame todos los comandos ejecutados por la sesión con ID xyz»—, Elasticsearch está optimizado para consultas analíticas exploratorias —«¿cuáles son las diez IPs más activas en las últimas veinticuatro horas?», «¿cómo se distribuye la actividad a lo largo del día?», «¿qué hashes aparecen con mayor frecuencia?»—. Esta complementariedad entre los dos sistemas de almacenamiento justifica el uso de ambos en paralelo: PostgreSQL para la persistencia relacional y la trazabilidad, Elasticsearch para la analítica rápida y la visualización.

2.6.3 Kibana como entorno de visualización analítica

Kibana es la interfaz de visualización del stack Elastic, diseñada para crear dashboards interactivos sobre datos indexados en Elasticsearch. Permite construir visualizaciones de múltiples tipos —gráficos de barras, gráficos de dona, tablas de datos, mapas de calor, paneles de métricas numéricas, histogramas temporales, mapas geográficos— y organizarlas en dashboards que pueden filtrarse interactivamente por cualquier campo de los datos subyacentes. La capacidad de filtrar el dashboard completo haciendo clic en un elemento —por ejemplo, hacer clic en una IP específica en el gráfico de Top IPs para ver todos los eventos asociados a esa IP en todos los paneles del dashboard— transforma la exploración de datos en una experiencia intuitiva que no requiere conocimientos de SQL o de los lenguajes de consulta de Elasticsearch. En el contexto de esta investigación, la visualización analítica de eventos de honeypots mediante Kibana representa un salto cualitativo respecto de la revisión directa de archivos de log. Mientras que la lectura de un archivo NDJSON exige que el analista reconstruya manualmente los patrones a partir de líneas de texto individuales, un dashboard

bien diseñado permite que esos patrones se vuelvan visibles de forma inmediata. Un pico en el histograma temporal puede indicar un momento de actividad intensa; una IP con una barra desproporcionadamente extensa en el gráfico de principales orígenes puede señalar un atacante especialmente activo; y una distribución de comandos dominada por instrucciones como `wget` puede sugerir intentos recurrentes de descarga de herramientas o archivos adicionales.

Esta capacidad de transformar registros textuales dispersos en representaciones visuales filtrables constituye el valor central de Kibana dentro del pipeline desarrollado.

2.7 Docker como tecnología de contenedorización transversal

Docker es una plataforma de contenedorización de código abierto que permite empaquetar aplicaciones y todas sus dependencias —bibliotecas, configuraciones, variables de entorno, archivos de sistema— en unidades autónomas denominadas contenedores. A diferencia de las máquinas virtuales, que emulan hardware completo y ejecutan sistemas operativos completos con sus kernels, los contenedores comparten el kernel del sistema operativo del anfitrión y solo aíslan el espacio de usuario, lo que los hace sustancialmente más ligeros, eficientes y rápidos de iniciar. En el contexto de este proyecto, Docker cumple una función transversal de infraestructura que hace posibles varias propiedades clave del sistema. La reproducibilidad: todos los componentes del sistema están definidos en imágenes Docker que pueden descargarse desde Docker Hub y ejecutarse en cualquier máquina con Docker instalado, favoreciendo que el entorno sea equivalente y repetible en cualquier máquina compatible donde se despliegue, siempre que se respeten las versiones, imágenes y variables de configuración documentadas. El aislamiento: cada servicio —Cowrie, `n8n`, PostgreSQL, Elasticsearch, Kibana— corre en su propio contenedor con sus propias dependencias, sin interferir con los demás servicios ni con el sistema anfitrión. La portabilidad: el mismo conjunto de contenedores puede ejecutarse en Windows, Linux o macOS sin modificaciones. La escalabilidad: si en el futuro se quisiera agregar más instancias de Cowrie o escalar Elasticsearch, Docker facilita esa operación.

2.8 Métricas de evaluación en sistemas de detección de intrusiones

La evaluación rigurosa de un sistema de detección de intrusiones requiere la definición previa de métricas que permitan traducir el comportamiento observado del sistema en indicadores comparables y objetivables. Las métricas más relevantes en el contexto de este proyecto, junto con su definición operacional y su relación con los resultados obtenidos, se presentan a continuación. El MTDD —Mean Time To Detect, Tiempo Medio de Detección— es la métrica central de evaluación de cualquier sistema de detección de amenazas. Se define como el promedio del tiempo transcurrido entre la ocurrencia de un evento de seguridad y el momento en que ese evento es detectado —es decir, queda disponible para el análisis por parte del operador—. En el contexto de este proyecto, el MTDD está fundamentalmente determinado por el intervalo de activación del Schedule Trigger de `n8n`: dado que el trigger se activa cada sesenta segundos, en el peor caso un evento puede tardar hasta sesenta segundos en ser detectado, y en el mejor caso puede ser detectado casi inmediatamente si ocurre justo antes de la activación del trigger. El MTDD promedio se considera, por tanto, un valor estimado de aproximadamente

treinta segundos, correspondiente a la mitad del intervalo de activación del Schedule Trigger. Esta métrica no constituye una medición empírica exacta del tiempo transcurrido entre cada evento y su alerta, ya que para ello sería necesario registrar de manera instrumental el timestamp exacto de generación del evento y el timestamp exacto de entrega de la notificación. La tasa de automatización expresa la proporción de las etapas del flujo de procesamiento del evento que se ejecutan sin ninguna intervención manual del operador, una vez que el sistema está en operación. En el sistema desarrollado en este proyecto, la tasa de automatización es del cien por ciento para todos los eventos dentro del alcance: desde que Cowrie registra el evento hasta que la alerta llega al celular del operador con geolocalización y resultado de VirusTotal, ningún paso requiere intervención humana. La trazabilidad del evento puede definirse como la capacidad del sistema de reconstruir el recorrido completo de un evento específico —desde su generación en el sensor hasta su almacenamiento y visualización— a partir de

un identificador único. En el sistema desarrollado, la trazabilidad se garantiza mediante el campo session de Cowrie, que permite relacionar todos los eventos de una misma sesión en PostgreSQL, Elasticsearch y Kibana, y la presencia del session ID en las alertas de Telegram, que permite vincular cada alerta con los registros correspondientes en la base de datos y en el dashboard.

Métrica	Definición	Operacional	Valor obtenido	Condición de medición
MTTD promedio	Tiempo medio entre ocurrencia y alerta	~30 segundos	Experimento controlado	40 sesiones
MTTD máximo	Tiempo máximo entre ocurrencia y alerta	~60 segundos	Intervalo del Schedule Trigger	Tasa de automatización
	Proporción del flujo sin intervención manual	100%	Todas las etapas del pipeline	Eventos capturados
	Total de eventos en PostgreSQL	219 eventos	Experimento de 40 sesiones, ~2 min	Sesiones únicas
	Conexiones SSH distintas identificadas	26 sesiones	Experimento de 40 sesiones	Alertas generadas
	Notificaciones Telegram enviadas	26 alertas	Sin intervención humana	IPs externas reales (24hs)
	IPs distintas capturadas en VPS	4+ IPs	Primeras 24hs de operación	VPS Eventos en VPS (24hs)
	Total eventos acumulados en VPS	>400 eventos	Primeras 24hs de operación	VPS Costo mensual de operación
	Costo total del sistema en producción	USD 6/mes	Solo el VPS; resto gratuito.	

Capítulo 3: Marco Metodológico

3.1 Naturaleza y tipo de investigación

La presente investigación es de naturaleza aplicada: su objetivo no es producir conocimiento teórico abstracto sobre honeypots, automatización de flujos o inteligencia de amenazas, sino diseñar, implementar y validar un artefacto técnico concreto —un laboratorio automatizado de honeypots— que resuelva un problema identificado en el contexto de la enseñanza de ciberseguridad en la Tecnicatura Universitaria en Programación. Esta orientación aplicada es coherente con la tradición de investigación en ingeniería de software y sistemas de información, donde la construcción y validación de artefactos —sistemas, herramientas, arquitecturas, metodologías— constituye la forma predominante de producción de conocimiento. Al mismo tiempo, el trabajo puede caracterizarse como de tipo descriptivo y evaluativo: describe con detalle la arquitectura construida, el proceso de implementación seguido y el comportamiento observado del sistema en condiciones controladas y reales, y evalúa ese comportamiento a la luz de un conjunto de métricas operativas previamente definidas. La dimensión descriptiva es necesaria para que la investigación sea reproducible: sin una descripción suficientemente detallada del sistema y del proceso de construcción, ningún otro equipo podría replicar los resultados. La dimensión evaluativa es necesaria para que la investigación sea académicamente significativa: sin criterios de evaluación explícitos y métricas definidas, la afirmación de que el sistema «funciona» no tendría contenido verificable. Es importante señalar explícitamente que la investigación no adopta un diseño experimental en el sentido estadístico estricto del término. No existe una asignación aleatoria de condiciones, no se manipula una variable independiente con control de las restantes, y los resultados no se analizan mediante pruebas de significancia estadística. El diseño es, en cambio, el de un estudio de caso con validación funcional: se construye un sistema específico en un contexto específico, se lo somete a condiciones de prueba definidas y se evalúa su comportamiento frente a criterios predeterminados. Este diseño es estándar y ampliamente aceptado en la investigación aplicada en ingeniería de software, sistemas de información y ciberseguridad cuando el objeto de estudio es un artefacto técnico construido por los propios investigadores.

3.2 Enfoque metodológico mixto

El enfoque metodológico adoptado en esta investigación puede caracterizarse como mixto, con predominio cualitativo y componente cuantitativo instrumental. Esta precisión es importante porque el trabajo no combina ambos enfoques en igualdad de peso ni busca realizar inferencias estadísticas generalizables, sino que articula la descripción, justificación e interpretación técnica del artefacto construido con la medición funcional de determinados resultados observables.

La dimensión cualitativa es predominante porque la investigación se centra en el diseño, implementación y documentación de una arquitectura técnica. Las principales decisiones del proyecto —selección de Cowrie como honeypot SSH, elección de Docker como plataforma de despliegue, uso de n8n como motor de automatización, adopción de PostgreSQL como capa de persistencia estructurada, integración de Elasticsearch y Kibana para indexación y visualización, incorporación de VirusTotal e ip-api.com para enriquecimiento, y uso de Telegram como canal de alertas— no pueden reducirse únicamente a variables numéricas. Cada una de estas decisiones requiere argumentación técnica, comparación de alternativas, análisis de restricciones, evaluación de compatibilidad entre componentes y documentación de problemas encontrados durante la implementación.

En este sentido, la dimensión cualitativa se expresa en la descripción del proceso de desarrollo, en la fundamentación de la arquitectura, en el análisis de las decisiones de diseño, en el registro de errores y soluciones, y en la interpretación del valor pedagógico y operativo del sistema. Esta dimensión permite comprender no solo qué resultado se obtuvo, sino también cómo se llegó a él, por qué se eligieron determinados componentes y qué condiciones técnicas hicieron posible la integración del pipeline.

La dimensión cuantitativa, por su parte, cumple una función instrumental de validación funcional. El sistema genera datos observables que permiten verificar su comportamiento bajo condiciones controladas y reales: cantidad de eventos procesados, número de sesiones identificadas, cantidad de alertas generadas, tipos de evento almacenados, volumen de registros incorporados a PostgreSQL, presencia de documentos indexados en Elasticsearch, resultados visualizados en Kibana, cantidad de eventos provenientes del VPS y estimación del tiempo medio de detección a partir del intervalo de ejecución del Schedule Trigger de n8n. Estas métricas no se utilizan para realizar pruebas de significancia estadística, sino para comprobar que las rutas principales del pipeline funcionan de acuerdo con los objetivos definidos.

La integración entre ambas dimensiones se produce de manera secuencial y complementaria. Primero, la dimensión cualitativa orienta el diseño del sistema y define qué arquitectura se construye, qué componentes se integran y qué comportamiento se espera de cada etapa. Luego, la dimensión cuantitativa permite observar si ese comportamiento efectivamente ocurre durante las pruebas. Finalmente, los resultados cuantitativos son interpretados nuevamente desde una perspectiva cualitativa, considerando el alcance del laboratorio, las limitaciones técnicas, la finalidad académica del proyecto y el carácter funcional —no estadístico— de la validación.

Por ejemplo, la decisión cualitativa de utilizar n8n como motor de automatización se valida cuantitativamente al observar que los eventos generados por Cowrie son procesados, normalizados, almacenados, indexados y enviados como alertas sin intervención manual dentro del alcance definido. Del mismo modo, la decisión cualitativa de incorporar VirusTotal e ip-api.com se verifica mediante la presencia efectiva de información de reputación de hashes y geocalización de IPs en las alertas generadas. En ambos casos, los datos numéricos no sustituyen el análisis técnico, sino que lo complementan y permiten comprobar que la arquitectura propuesta opera de forma funcional.

Por lo tanto, el enfoque mixto de esta investigación no debe entenderse como una combinación estadística tradicional de métodos cualitativos y cuantitativos, sino como una estrategia metodológica adecuada para un trabajo de desarrollo tecnológico: el componente cualitativo permite diseñar, justificar y comprender el artefacto, mientras que el componente cuantitativo permite verificar su funcionamiento mediante indicadores observables.

3.3 Diseño de investigación: laboratorio con validación funcional incremental

El diseño adoptado para esta investigación es el de laboratorio con validación funcional incremental. En este diseño, el sistema se construye en fases sucesivas donde cada fase añade una nueva capa de funcionalidad sobre la base validada de la fase anterior, y el avance hacia la siguiente fase está condicionado a la verificación positiva del funcionamiento de la fase actual. Este diseño responde a un criterio metodológico de robustez: construir sobre fundamentos verificados en lugar de construir todo el sistema y validarlo al final. El diseño incremental tiene además ventajas pedagógicas y documentales. Al construir y validar el sistema capa por capa, es posible aislar y diagnosticar problemas en cada etapa sin la complejidad de un sistema completamente integrado que puede fallar por razones difíciles de identificar.

La documentación de cada fase —incluyendo los errores encontrados y las soluciones adoptadas— produce un registro del proceso de construcción que tiene valor formativo propio: muestra cómo se resuelven problemas técnicos reales en el contexto de un proyecto de esta naturaleza, lo que ningún tutorial idealizado podría replicar.

Las nueve fases de implementación del proyecto, con su lógica de dependencia incremental, se presentan en la siguiente tabla:

Fase	Contenido principal	Criterio de validación
Fase 1: Diseño y planificación	Definición de la arquitectura, selección de herramientas y diseño del esquema de base de datos	Documento de diseño aprobado por tutores
Fase 2: Despliegue de Cowrie local	Instalación de Docker, despliegue de Cowrie, verificación de captura de eventos	Cowrie responde a conexiones SSH y genera logs NDJSON
Fase 3: Persistencia PostgreSQL	Creación del esquema de BD, configuración del contenedor PostgreSQL	Eventos insertados correctamente y consultables
Fase 4: Workflow n8n	Desarrollo del pipeline SOAR-lite con parser NDJSON y control de timestamps	Eventos procesados automáticamente cada 60 segundos
Fase 5: ELK Stack y Kibana	Integración de Elasticsearch, configuración de dashboards en Kibana	Dashboards muestran eventos en tiempo real
Fase 5B: Pruebas con Hydra	Simulación de ataques desde VM Ubuntu con Hydra, recepción de alertas Telegram	Sistema detecta y alerta ataques de fuerza bruta
Fase 6: Experimento 40 sesiones	Experimento controlado con Hydra + sshpass, cálculo de métricas	219 eventos registrados, 26 alertas, MTTD ~30s
Fase 7: VPS y exposición real	Aprovisionamiento VPS, despliegue Cowrie con IP pública, sincronización de logs	Ataques reales capturados en <30 min del despliegue
Fase 8: Mejoras de enriquecimiento	Integración VirusTotal + ip-api.com, alertas con geolocalización y reputación	Alertas incluyen país, ciudad, ISP y resultado VT

3.4 Hipótesis de investigación

3.4.1 Hipótesis principal

La hipótesis principal de la investigación sostiene que la integración de un honeypot de media interacción basado en Cowrie con un pipeline automatizado compuesto por n8n, PostgreSQL, Elasticsearch, Kibana, Telegram y APIs externas de inteligencia de amenazas permite mejorar funcionalmente la gestión de eventos de intrusión SSH respecto de una revisión manual periódica de archivos de log, dentro del alcance definido para este trabajo.

A efectos de esta investigación, la mejora funcional se considera verificable si el sistema demuestra capacidad para cumplir de manera automática las siguientes condiciones: leer y parsear eventos generados por Cowrie en formato NDJSON; normalizar los campos relevantes de cada evento; almacenar los registros en PostgreSQL preservando la trazabilidad de las sesiones; indexar los eventos en Elasticsearch; visualizarlos en Kibana; clasificar eventos críticos; enriquecerlos con información externa de geolocalización de IPs y reputación de hashes cuando corresponda; y generar alertas por Telegram sin intervención humana durante el procesamiento.

Esta hipótesis no presupone una mejora estadísticamente demostrada frente a un grupo de comparación manual ni una medición directa del tiempo real evento-alerta. La reducción de latencia operativa se interpreta como una mejora funcional esperada del diseño automatizado frente a una revisión manual periódica, mientras que el MTTR de aproximadamente treinta segundos se considera una estimación derivada del intervalo de ejecución del Schedule Trigger de n8n. Por lo tanto, la hipótesis se contrasta mediante la verificación del funcionamiento integral del pipeline y no mediante una prueba estadística formal.

3.4.2 Hipótesis secundaria sobre reproducibilidad potencial

La primera hipótesis secundaria sostiene que la documentación técnica detallada del proceso de implementación favorece la reproducibilidad potencial del laboratorio OSCORP por parte de otros equipos académicos con conocimientos equivalentes a los de un egresado de la Tecnicatura Universitaria en Programación.

A efectos de esta investigación, la reproducibilidad no se entiende como una propiedad empíricamente validada mediante la replicación externa del sistema por terceros, sino como una condición metodológica favorecida por la existencia de documentación técnica suficiente, ordenada y trazable. Esta documentación incluye la arquitectura general del sistema, el esquema de base de datos, el código del parser NDJSON, los scripts de sincronización, el script del experimento controlado, el workflow de n8n, las evidencias de funcionamiento del stack Docker, los dashboards de Kibana y los resultados obtenidos durante las pruebas.

La hipótesis se considera respaldada funcionalmente si el trabajo documenta de manera clara las fases de implementación, las decisiones de diseño, los errores encontrados, las soluciones adoptadas y los componentes técnicos necesarios para reconstruir el laboratorio. Sin embargo, su validación empírica completa requeriría que un equipo externo intente reproducir el sistema a partir de la documentación provista, registrando dificultades, ajustes necesarios y grado de éxito alcanzado. Esa replicación externa queda fuera del alcance de la presente tesis y se propone como línea de trabajo futuro.

3.4.3 Hipótesis secundaria sobre riqueza analítica

La tercera hipótesis sostiene que los logs generados por Cowrie en condiciones normales de operación contienen evidencia suficientemente rica y estructurada para identificar comportamientos relevantes de los atacantes —incluyendo credenciales utilizadas, comandos de reconocimiento ejecutados, herramientas descargadas y sus hashes SHA-256— y que esta evidencia es suficiente para alimentar un sistema de inteligencia de amenazas automatizado capaz de generar alertas con valor analítico real para el operador.

3.4.4 Hipótesis secundaria sobre tráfico real

La cuarta hipótesis sostiene que la exposición de un honeypot SSH correctamente configurado a internet real mediante un servidor VPS con IP pública directa generará tráfico malicioso autónomo proveniente de bots y atacantes externos en un período de tiempo inferior a una hora desde el momento de la activación, sin necesidad de ninguna acción de atracción o publicidad del servicio, validando así el sistema con condiciones de operación reales y no solo simuladas.

3.5 Variables e indicadores

3.5.1 Variable independiente principal

La variable independiente principal de la investigación es la presencia o ausencia de un pipeline de automatización para el tratamiento de los eventos generados por el honeypot. Esta variable define la diferencia sustantiva entre los dos escenarios que la investigación compara: el escenario de referencia, donde los eventos de Cowrie permanecen en el archivo `cowrie.json` y su análisis depende de la revisión manual periódica por parte del operador, y el escenario propuesto, donde los mismos eventos son procesados automáticamente por el pipeline de `n8n` y quedan disponibles en PostgreSQL, Elasticsearch y Kibana en un plazo máximo de sesenta segundos, con alertas enviadas automáticamente a Telegram para los eventos de mayor criticidad.

3.5.2 Variables dependientes

Las variables dependientes de la investigación son las métricas de desempeño del sistema que permiten evaluar el impacto de la variable independiente sobre la calidad del monitoreo de amenazas. Se definen cinco variables dependientes principales, cada una con su indicador operacional específico:

La primera variable dependiente es el tiempo medio de detección, que se operacionaliza como el tiempo promedio transcurrido entre el momento en que Cowrie registra un evento crítico y el momento en que la alerta correspondiente es entregada al operador mediante Telegram. Su indicador es el tiempo en segundos, y el valor esperado —basado en el intervalo del Schedule Trigger— es de treinta segundos en promedio y sesenta segundos en el peor caso.

La segunda variable dependiente es la tasa de automatización, que se operacionaliza como la proporción de las etapas del flujo de procesamiento —lectura, parseo, normalización, almacenamiento, indexación, enriquecimiento y alertado— que se ejecutan sin intervención manual del operador. Su indicador es un porcentaje entre cero y cien, y el valor esperado es del cien por ciento para todas las etapas dentro del alcance.

La tercera variable dependiente es la trazabilidad del evento, que se operacionaliza como la capacidad de reconstruir el recorrido completo de un evento específico desde el archivo de logs de Cowrie hasta la alerta de Telegram, pasando por PostgreSQL y Elasticsearch, utilizando el campo session como identificador común. Su indicador es la presencia o ausencia del campo session en cada etapa del pipeline.

La cuarta variable dependiente es la riqueza analítica del registro, que se operacionaliza como el número de campos de información disponibles sobre cada evento después del procesamiento por el pipeline, incluyendo los campos nativos de Cowrie más los campos de enriquecimiento aportados por VirusTotal e ip-api.com. Su indicador es el número de campos únicos disponibles en el registro final.

La quinta variable dependiente es el costo de operación del sistema, que se operacionaliza como el costo monetario mensual de mantener el sistema en operación continua. Su indicador es el costo en dólares estadounidenses por mes, y el valor obtenido es de seis dólares mensuales correspondientes al servidor VPS, con todos los demás componentes completamente gratuitos.

3.6 Técnicas e instrumentos de recolección de datos

La técnica principal de recolección de datos en esta investigación es la observación técnica automatizada de eventos generados por el honeypot. Esta observación se realiza en tiempo real a través del archivo cowrie.json generado por Cowrie, que constituye el instrumento primario de captura de evidencia. Como instrumentos complementarios se utilizan las tablas de PostgreSQL —que permiten verificar la correcta inserción y estructura de los eventos—, los dashboards de Kibana —que permiten verificar la correcta indexación y visualización—, las ejecuciones del workflow de n8n —que permiten auditar el comportamiento del pipeline—, y los mensajes de Telegram —que permiten verificar el correcto funcionamiento del subsistema de alertas—.

Para la validación controlada del sistema se diseñó el experimento de cuarenta sesiones descrito en la sección de desarrollo, que utilizó dos herramientas de ataque desde una VM Ubuntu en VirtualBox: Hydra para la simulación de ataques de fuerza bruta SSH con un diccionario de diez contraseñas comunes, y sshpass para la automatización de sesiones SSH completas con

comandos de reconocimiento predefinidos. El script del experimento —`experimento_40.sh`— está disponible en el repositorio GitHub del proyecto y puede ser utilizado por otros investigadores para reproducir el experimento en condiciones equivalentes. Para la validación con tráfico real, el instrumento es el propio VPS expuesto a internet, que actúa como sensor pasivo que recibe sin ninguna intervención activa de los investigadores el tráfico malicioso generado de forma autónoma por bots de internet. El período de observación inicial fue de veinticuatro horas desde la activación del VPS, lo que permitió capturar más de cuatrocientos eventos de tráfico real.

3.7 Procedimiento de investigación

El procedimiento de investigación siguió la secuencia de nueve fases de implementación incremental descrita en la sección de diseño.

En cada fase, el procedimiento estándar consistió en: primero, diseñar la nueva funcionalidad a implementar y documentar las decisiones de diseño previas a la implementación; segundo, implementar la funcionalidad en el entorno de laboratorio; tercero, ejecutar pruebas de verificación funcional para confirmar que la nueva funcionalidad opera correctamente; cuarto, documentar el proceso de implementación incluyendo los errores encontrados y las soluciones adoptadas; quinto, solo una vez verificado el correcto funcionamiento, avanzar a la siguiente fase. Este procedimiento iterativo y documentado garantiza que el avance del proyecto esté siempre fundamentado en verificaciones positivas de funcionamiento, no en suposiciones sobre el correcto desempeño de componentes no verificados. También garantiza la trazabilidad del proceso de construcción: la documentación de cada fase permite que cualquier lector del informe pueda entender no solo qué sistema fue construido sino también cómo fue construido, qué decisiones se tomaron y por qué, y qué problemas se encontraron y cómo se resolvieron.

3.8 Consideraciones éticas, legales y de seguridad

Las investigaciones en el ámbito de la ciberseguridad están sujetas a consideraciones éticas y legales específicas que esta tesis aborda explícitamente.

El principio rector que guía todas las decisiones de diseño y operación del sistema desarrollado es el de la observación pasiva en entorno controlado: el laboratorio registra actividad maliciosa que los atacantes dirigen voluntariamente hacia él, pero no toma ninguna acción ofensiva hacia los atacantes ni hacia terceros. En el caso del honeypot local, el sistema opera completamente dentro de la red doméstica de los investigadores, sin exposición a internet.

Las interacciones que registra son las generadas por los propios investigadores durante las pruebas controladas. No existe ningún riesgo de captura de tráfico de terceros no autorizados. En el caso del honeypot en el VPS, el sistema es expuesto deliberadamente a internet con el propósito de capturar tráfico malicioso autónomo.

Este tipo de despliegue es una práctica estándar y ampliamente aceptada en la investigación académica de ciberseguridad, siempre que se implementen las siguientes salvaguardas, todas

presentes en este proyecto: el honeypot está completamente aislado dentro de un contenedor Docker que impide que un atacante que «comprometa» el entorno emulado pueda acceder al sistema operativo real del VPS ni a ningún recurso fuera del contenedor; el servicio emulado por Cowrie no procesa transacciones reales, no almacena datos de usuarios reales y no tiene acceso a ningún sistema de información con datos sensibles; el VPS no tiene credenciales de acceso a ningún sistema externo que pudiera ser comprometido si el VPS en sí fuera vulnerado; el tráfico malicioso capturado se utiliza exclusivamente con fines de investigación académica y no es publicado en formas que pudieran comprometer la privacidad de los atacantes como individuos.

Respecto del uso de las APIs de VirusTotal e ip-api.com: ambas APIs son utilizadas dentro de los términos de servicio correspondientes. La cuenta gratuita de VirusTotal se obtiene mediante registro en la plataforma y su uso está explícitamente autorizado para investigación académica no comercial dentro de los límites de cuota definidos. El servicio ip-api.com ofrece explícitamente acceso gratuito para uso no comercial en su documentación pública.

Capítulo 4: Desarrollo de la Investigación

4.1 Arquitectura general del sistema

La arquitectura del laboratorio OSCOP está organizada en dos planos operativos complementarios que se integran mediante un mecanismo de sincronización automática. El plano de captura, ubicado en el servidor VPS en Amsterdam, es el componente que opera de forma permanente e independiente: el honeypot Cowrie corre continuamente, expuesto a internet con IP pública directa, registrando en su archivo de logs todo el tráfico malicioso que llega de forma autónoma desde bots y atacantes distribuidos globalmente. El plano de procesamiento y análisis, ubicado en la PC local de los investigadores, se activa cuando esta está encendida: el script de sincronización descarga periódicamente el archivo de logs del VPS mediante SCP, y el pipeline de n8n procesa los eventos nuevos, los almacena en PostgreSQL y Elasticsearch, los enriquece con VirusTotal e ip-api.com, y genera alertas de Telegram para los eventos críticos. Esta arquitectura híbrida —VPS para la captura continua, PC local para el procesamiento— responde a una restricción práctica identificada durante el proyecto: el plan básico del VPS en DigitalOcean (1 GB de RAM) no tiene capacidad suficiente para ejecutar simultáneamente Cowrie, n8n, PostgreSQL, Elasticsearch y Kibana, ya que Elasticsearch por sí solo requiere al menos 512 MB de RAM en su configuración mínima y tiende a consumir más en operación normal. La solución adoptada —ejecutar solo Cowrie en el VPS y el resto del stack en la PC local— mantiene la capacidad de captura continua de tráfico real mientras aprovecha los recursos de hardware disponibles en la PC local para el procesamiento analítico.

La siguiente tabla resume todos los componentes del sistema con su función, tecnología, ubicación y acceso:

Componente	Función en el sistema	Tecnología	Ubicación	Acceso
Honeypot SSH	Captura ataques reales 24/7	Cowrie / Docker	VPS Amsterdam (167.71.8.253)	Puerto 2222 público
Sincronización de logs	Descarga cowrie.json del VPS cada 55s	Script PowerShell + SCP	PC local (automático)	Clave SSH RSA 4096 bits
Honeypot SSH local	Captura ataques en pruebas locales	Cowrie / Docker	PC local	Puerto 2222 local
Motor de automatización	Pipeline SOAR-lite orquestador	n8n / Docker	PC local	http://localhost:5678
Base de datos relacional	Persistencia estructurada de eventos	PostgreSQL / Docker	PC local	Puerto 5433

Motor de indexación	Indexación analítica para Kibana	Elasticsearch / Docker	PC local	Puerto 9200
Visualización analítica	Dashboards interactivos de ataques	Kibana / Docker	PC local	http://localhost:5601
Análisis de malware	Consulta reputación de hashes SHA-256	API VirusTotal v3	Cloud externo (gratuito)	HTTPS REST API
Geolocalización de IPs	País, ciudad e ISP del atacante	API ip-api.com	Cloud externo (gratuito)	HTTP REST API
Notificaciones en tiempo real	Alertas enriquecidas al celular	Bot de Telegram	Dispositivo móvil	Bot API de Telegram
VM de ataque controlado	Generación de ataques en experimentos	Ubuntu / VirtualBox	PC local	Red interna

4.2 Fase 1 y 2: Diseño del entorno y despliegue de Cowrie

4.2.1 Configuración del entorno local con Docker

La primera fase del proyecto consistió en establecer el entorno de desarrollo local sobre el que se construirían todas las fases posteriores. La decisión de usar Docker como plataforma de contenedorización transversal fue tomada desde el inicio del proyecto por las razones de reproducibilidad, aislamiento y portabilidad descritas en el marco teórico. El entorno local consiste en una PC con Windows 11 y Docker Desktop instalado, que actúa como anfitrión para todos los contenedores del sistema.

El comando de verificación que confirma el correcto funcionamiento del entorno Docker es `docker ps`, que lista los contenedores activos. En el estado operativo normal del proyecto, este comando debe mostrar los cinco contenedores del sistema: `friendly_kalam` —el contenedor de Cowrie local, mapeado al puerto 2222—, `db-honeypot` —el contenedor de PostgreSQL, mapeado al puerto 5433—, `n8n` —el contenedor del motor de automatización, mapeado al puerto 5678—, `elasticsearch` —el contenedor del motor de indexación, mapeado al puerto 9200—, y `kibana` —el contenedor de visualización, mapeado al puerto 5601—.

4.2.2 Despliegue de Cowrie en el entorno local

El despliegue de Cowrie en el entorno local se realizó utilizando la imagen oficial `cowrie/cowrie` disponible en Docker Hub. El comando de despliegue incluye tres parámetros críticos: el mapeo de puertos `-p 2222:2222` que expone el servicio SSH emulado en el puerto 2222 de la interfaz de red local, el flag `--restart always` que garantiza que el contenedor se reinicie automáticamente si Docker se reinicia, y el montaje de volumen `-v` que mapea la carpeta

de logs del contenedor hacia una carpeta del sistema anfitrión, permitiendo que n8n lea los logs generados por Cowrie desde el sistema de archivos de Windows. La verificación del correcto funcionamiento de Cowrie se realizó conectándose al honeypot desde la misma PC mediante el cliente SSH de PowerShell: `ssh root@localhost -p 2222`. La respuesta de Cowrie —una bienvenida del sistema Debian simulado— confirmó que el servicio estaba operativo. La inspección del archivo `cowrie.json` generado confirmó que los eventos de conexión y autenticación se estaban registrando correctamente con todos los campos esperados.

4.3 Fase 3: Diseño e implementación de la base de datos PostgreSQL

El diseño del esquema de base de datos en PostgreSQL fue una de las decisiones técnicas más importantes del proyecto, ya que determina qué información queda disponible para las consultas analíticas y qué relaciones pueden establecerse entre los eventos de una misma sesión. El esquema adoptado refleja el compromiso entre la completitud —capturar todos los campos relevantes de Cowrie— y la simplicidad —mantener una tabla única que facilite las consultas sin necesidad de joins complejos—. La tabla `eventos` contiene los siguientes campos: un identificador serial autoincremental como clave primaria, el timestamp del evento con zona horaria, la IP de origen del atacante, el nombre de usuario utilizado en la autenticación, la contraseña utilizada en la autenticación, el comando ejecutado en la shell simulada, el hash SHA-256 del archivo descargado, el identificador de sesión de Cowrie, el identificador de tipo de evento de Cowrie, la URL desde la que se descargó un archivo, y un campo de hash de integridad del evento. La presencia simultánea del campo `session` y del campo `eventid` permite realizar los dos tipos de consultas más importantes: consultas por sesión —«¿qué ocurrió en la sesión xyz?»— y consultas por tipo de evento —«¿cuántos intentos de autenticación fallidos hubo en las últimas veinticuatro horas?»—. Un problema técnico que se presentó durante la implementación y que merece documentación detallada por su impacto metodológico fue el desajuste de zona horaria entre los timestamps generados por Cowrie —que usa UTC— y el entorno de la PC local —que usa la zona horaria `America/Argentina/Buenos_Aires, UTC-3`—. Este desajuste se manifestaba en consultas SQL con condiciones sobre el campo `timestamp` que devolvían resultados incorrectos o vacíos cuando se utilizaba la función `NOW()` sin conversión de zona horaria. La solución adoptada fue utilizar el tipo `TIMESTAMPTZ` en PostgreSQL —que almacena siempre en UTC y convierte automáticamente según la zona horaria del cliente— y especificar explícitamente la zona horaria en las consultas con condiciones temporales.

4.4 Fase 4: Desarrollo del workflow de automatización en n8n

4.4.1 Estructura general del workflow

El workflow de n8n constituye el núcleo funcional del sistema SOAR-lite y representa la contribución técnica más original del proyecto. Su desarrollo requirió resolver varios problemas que no tienen solución directa en la documentación estándar de n8n: el parseo línea por línea de archivos NDJSON en el nodo `Code` de JavaScript, el control de timestamps para evitar el reprocesamiento de eventos, el acceso a datos de nodos anteriores en el flujo cuando el contexto

del nodo actual corresponde a los datos transformados por una API intermedia, y el enrutamiento diferenciado de eventos según su tipo para aplicar el enriquecimiento de VirusTotal solo a los eventos de descarga de archivo. El workflow completo consta de doce nodos organizados en una estructura de árbol con dos ramas diferenciadas. La rama principal procesa todos los eventos y los almacena en PostgreSQL y Elasticsearch. La rama de alertas procesa solo los eventos de mayor criticidad —logins exitosos y descargas de archivos— y genera las notificaciones de Telegram enriquecidas. Dentro de la rama de alertas, una subbifurcación implementada mediante un nodo IF separa los eventos de descarga —que requieren consulta a VirusTotal— de los eventos de login —que no la requieren—.

Tabla de nodos del workflow n8n

Nodo	Tipo	Función en el pipeline
Trigger cada 60 seg	Schedule Trigger	Activa el workflow automáticamente cada 60 segundos sin intervención del operador
Leer cowrie_vps.json	Read/Write Files from Disk	Lee el archivo de logs NDJSON sincronizado desde el VPS mediante el script de SCP
Parser NDJSON + Timestamp	Code (JavaScript)	Parsea el archivo línea por línea, filtra solo eventos nuevos mediante control de lastTimestamp y devuelve los eventos no procesados
Filtrar eventos Cowrie	Filter	Filtra únicamente los eventos cuyo campo eventid contenga el prefijo 'cowrie', descartando líneas no válidas
Guardar en PostgreSQL	PostgreSQL	Inserta cada evento nuevo en la tabla eventos de la base de datos relacional para persistencia estructurada
Indexar en Elasticsearch	Elasticsearch	Crea un documento por cada evento en el índice de Elasticsearch para alimentar los dashboards de Kibana
Filtrar eventos críticos	Filter	Retiene únicamente los eventos de tipo cowrie.login.success y cowrie.session.file_download para el subsistema de alertas
Geolocalización (ip-api)	HTTP Request	Consulta la API de ip-api.com con la IP del atacante para obtener país, ciudad, ISP y coordenadas geográficas
¿Descarga de archivo?	IF	Bifurca el flujo: rama true para eventos de descarga (requieren VirusTotal) y rama false para logins exitosos
Consultar VirusTotal	HTTP Request	Consulta la API v3 de VirusTotal con el hash SHA-256 del archivo descargado para determinar si es reconocido como malware
Alerta Descarga + VT	Telegram	Envía al operador una alerta de descarga de archivo enriquecida con geolocalización y resultado de VirusTotal
Alerta Login Exitoso	Telegram	Envía al operador una alerta de login exitoso enriquecida con geolocalización completa del atacante

4.4.2 El nodo Code y el parser NDJSON

El nodo Code es el componente más complejo del workflow. Implementa en JavaScript la lógica de parseo del archivo NDJSON y el control de timestamps que constituye el mecanismo central de idempotencia del sistema —la garantía de que cada evento es procesado exactamente una vez—. El código comienza leyendo el contenido binario del archivo `cowrie.json` mediante la API de datos

binarios de `n8n`, lo convierte a texto UTF-8, y lo divide en líneas individuales usando el separador de nueva línea. El control de timestamps funciona mediante el mecanismo `$getWorkflowStaticData('global')` de `n8n`, que permite persistir datos entre ejecuciones sucesivas del workflow. En cada ejecución, el nodo lee el valor `lastTimestamp` almacenado en esta memoria persistente —o usa `'1970-01-01T00:00:00.000Z'` como valor por defecto si es la primera ejecución— y solo incluye en la salida los eventos cuyo timestamp sea estrictamente posterior a ese valor. Al final de la ejecución, actualiza el `lastTimestamp` con el timestamp más reciente de los eventos procesados en esa ejecución. Este mecanismo garantiza que cuando `n8n` lee el archivo `cowrie.json` —que acumula todos los eventos históricos desde el inicio de la operación de Cowrie— solo procese los eventos que son nuevos desde la última ejecución.

4.4.3 Integración con PostgreSQL y Elasticsearch

Los nodos de inserción en PostgreSQL y creación de documentos en Elasticsearch están conectados en paralelo a la salida del nodo Filter, de manera que cada evento que supere los criterios del filtro sea insertado simultáneamente en ambos sistemas. Esta arquitectura de escritura paralela garantiza que los datos estén disponibles en tiempo real tanto para las consultas relacionales de PostgreSQL como para las visualizaciones de Kibana. La integración con PostgreSQL utiliza el nodo nativo de PostgreSQL de `n8n`, configurado con la cadena de conexión correspondiente al contenedor `db-honeypot`. Los campos del documento insertado corresponden exactamente a las columnas de la tabla `eventos`: `timestamp`, `src_ip`, `username`, `password`, `command`, `shasum`, `session`, `eventid`, `url` y `hash`. Los campos que no están presentes en un tipo de evento particular —por ejemplo, `command` y `shasum` en un evento de tipo `cowrie.session.connect`— se insertan como `NULL`, lo que es manejado correctamente por el esquema de la tabla.

4.4.4 El subsistema de alertas: Filtro Atacantes, ip-api, IF, VirusTotal y

Telegram El subsistema de alertas es la parte del workflow que transforma los eventos técnicos en notificaciones comprensibles y accionables para el operador. Comienza con el nodo Filtro Atacantes, que aplica un filtro adicional sobre los eventos que ya superaron el Filter principal, reteniéndolos solo si el campo `eventid` corresponde a `cowrie.login.success` o `cowrie.session.file_download`. Estos dos tipos de evento son los de mayor criticidad operativa: el primero indica que un atacante logró autenticarse en el honeypot y tiene acceso a la shell simulada, el segundo indica que un atacante descargó un archivo —potencialmente un binario malicioso— desde el honeypot. Una vez que el Filtrar eventos críticos identifica un evento crítico, el flujo pasa inmediatamente al nodo HTTP Request de `ip-api.com`, que consulta la API

de geolocalización con la IP de origen del evento. Esta consulta se ejecuta para todos los eventos críticos, independientemente de su tipo, porque la información geográfica del atacante es valiosa para ambas categorías. La respuesta de ip-api.com incluye los campos country, city, isp, lat y lon, entre otros, todos los cuales quedan disponibles para los nodos siguientes. El nodo IF que sigue a la geolocalización implementa la bifurcación que separa los eventos de descarga — que requieren consulta a VirusTotal— de los eventos de login —que no la requieren—. La condición del IF evalúa el campo eventid del evento original, accedido mediante la sintaxis \$('Filtro Atacantes').item.json.eventid, para verificar si es igual a cowrie.session.file_download. Los eventos que cumplen esta condición pasan a la rama true, donde el nodo HTTP Request de VirusTotal consulta la API con el hash SHA-256 del archivo descargado. Los eventos que no cumplen la condición —los logins exitosos— pasan directamente a la rama false, donde el nodo Telegram correspondiente construye y envía la alerta de login.

4.5 Fase 5 y 5B: Integración del stack analítico y primeras

pruebas de ataque

4.5.1 Configuración de Elasticsearch y Kibana

La integración de Elasticsearch en el sistema fue directa en términos de configuración del contenedor, pero requirió ajustes específicos en los parámetros de arranque para garantizar un funcionamiento estable dentro de los recursos disponibles en la PC local. El parámetro más crítico fue la configuración de la memoria heap de la JVM mediante la variable de entorno ES_JAVA_OPTS, establecida en un valor de 512 megabytes para el heap mínimo y máximo, que garantiza que Elasticsearch no intente reservar más memoria de la disponible y al mismo tiempo tiene suficiente para operar con los volúmenes de datos del proyecto. La configuración de Kibana requirió crear un Data View —anteriormente denominado Index Pattern— que mapea el índice de Elasticsearch donde n8n inserta los eventos de Cowrie. Una vez creado el Data View, Kibana puede explorar los datos y construir visualizaciones sobre cualquier campo disponible. Los cuatro paneles del dashboard principal fueron configurados utilizando las capacidades de visualización nativa de Kibana sin necesidad de código adicional: el gráfico de dona de distribución de comandos utiliza una agregación de términos sobre el campo command.keyword, el panel de Top IPs utiliza una agregación de términos sobre src_ip.keyword, el histograma de actividad temporal utiliza una agregación de histograma de fechas sobre el campo timestamp con intervalo de doce horas, y el panel de indicadores de compromiso utiliza una agregación de términos sobre hash.keyword.

4.5.2 Primera campaña de pruebas con Hydra desde VM Ubuntu

La Fase 5B consistió en la primera campaña de ataques controlados utilizando Hydra desde una máquina virtual Ubuntu en VirtualBox. Esta fase tuvo dos objetivos simultáneos: por un lado, verificar el funcionamiento completo del pipeline de extremo a extremo bajo condiciones de carga real —múltiples eventos en poco tiempo—, y por otro lado, verificar que el sistema de alertas de

Telegram operaba correctamente generando notificaciones automáticas para los eventos de login exitoso detectados por Hydra. Hydra es una herramienta de fuerza bruta para protocolos de red que prueba sistemáticamente combinaciones de usuarios y contraseñas definidas en listas —denominadas diccionarios— hasta encontrar una combinación que produzca una autenticación exitosa. En el contexto de este proyecto, Hydra fue configurado para atacar el honeypot Cowrie en el puerto 2222 con el usuario root y un diccionario de diez contraseñas comunes. Dado que Cowrie acepta cualquier credencial como válida, todas las contraseñas del diccionario producen eventos `cowrie.login.success`, lo que permite verificar que el sistema genera correctamente una alerta de Telegram por cada login exitoso. Los resultados de esta primera campaña confirmaron el funcionamiento del pipeline completo: Hydra generó doce intentos de autenticación, todos exitosos desde la perspectiva de Cowrie, y el sistema generó dos alertas de Telegram —una por cada contraseña diferente que ingresó correctamente y generó un evento registrado en el archivo de logs antes del siguiente ciclo del trigger— dentro del período de sesenta segundos del Schedule Trigger. El dashboard de Kibana mostró el pico de actividad correspondiente al período de ejecución de Hydra, y DBeaver confirmó que los eventos estaban correctamente insertados en la tabla eventos de PostgreSQL.

4.6 Fase 6: Experimento controlado de cuarenta sesiones

4.6.1 Diseño del experimento

El experimento de cuarenta sesiones fue diseñado para generar un corpus de validación más rico y representativo que las pruebas de Hydra de la fase anterior, incorporando dos tipos de comportamiento de ataque: la fuerza bruta pura de credenciales, que genera principalmente eventos `cowrie.login.failed` y `cowrie.login.success`, y las sesiones completas con comandos de reconocimiento y descarga de archivos, que generan además eventos `cowrie.command.input` y `cowrie.session.file_download`.

La elección de cuarenta sesiones respondió a un criterio metodológico pragmático y funcional, no estadístico. El objetivo del experimento no fue construir una muestra probabilística ni generalizar patrones de ataque a una población externa, sino generar un volumen suficiente de eventos controlados para verificar el funcionamiento integral del pipeline bajo condiciones de carga moderada. Se definieron veinte intentos asociados a fuerza bruta con Hydra y veinte sesiones completas mediante `sshpas` para equilibrar dos tipos de comportamiento relevantes: la autenticación automatizada y la actividad post-autenticación dentro de la shell emulada. Esta distribución permitió producir eventos de conexión, login fallido, login exitoso, ejecución de comandos, descarga de archivo, generación de hash, almacenamiento en PostgreSQL, indexación en Elasticsearch, visualización en Kibana y alertado mediante Telegram. Al mismo tiempo, el número total de sesiones se mantuvo acotado para evitar una sobrecarga innecesaria del entorno local de laboratorio, de `n8n` y de los servicios de enriquecimiento externo utilizados en el pipeline.

Esta variedad de tipos de eventos permite validar todos los componentes del pipeline, incluyendo el nodo IF que bifurca según el tipo de evento y el nodo HTTP Request de VirusTotal que solo se activa para eventos de descarga. El experimento utilizó dos herramientas desde la VM Ubuntu: Hydra para la Fase 1 —veinte intentos de fuerza bruta con el diccionario de contraseñas común—, y sshpass para la Fase 2 —veinte sesiones completas con comandos predefinidos—. La herramienta sshpass permite automatizar la autenticación SSH pasando la contraseña como argumento de línea de comandos, lo que la hace adecuada para la automatización de sesiones SSH completas que incluyen la ejecución de comandos dentro de la shell. La Fase 2 del experimento usó comandos de reconocimiento típicos del comportamiento post-autenticación de los atacantes: `whoami` para obtener el usuario actual, `uname -a` para obtener información del sistema operativo, `cat /etc/passwd` para leer el archivo de usuarios, `ls` para listar el directorio actual, y `wget http://example.com/malware.sh` para simular la descarga de un archivo malicioso.

4.6.2 Ejecución y resultados

El experimento fue ejecutado desde la terminal de Ubuntu de la VM de VirtualBox mediante el script `experimento_40.sh`, que automatizó las cuarenta sesiones con una pausa de tres segundos entre cada sesión para garantizar el correcto registro de cada evento en Cowrie antes de iniciar la siguiente sesión. La ejecución completa del experimento tomó aproximadamente dos minutos, con el primer evento registrado a las 10:10:03 (hora Argentina) y el último a las 10:11:56. Los resultados del experimento, verificados mediante consultas SQL en DBeaver y en los dashboards de Kibana, fueron los siguientes: doscientos diecinueve eventos totales registrados en PostgreSQL, correspondientes a veintiséis sesiones únicas identificadas mediante el campo `session`.

El desfase entre las cuarenta sesiones diseñadas y las veintiséis sesiones únicas registradas no invalida la finalidad funcional del experimento, pero sí constituye una limitación metodológica que debe ser explicitada. La diferencia se explica principalmente por la forma en que Cowrie registra las conexiones SSH: no todo intento iniciado por una herramienta automatizada se consolida necesariamente como una sesión completa identificable mediante el campo `session`. En particular, algunos intentos de Hydra pueden ser rechazados o cerrados antes de completar el handshake SSH o antes de generar una secuencia completa de eventos asociada a una sesión persistente. Por ese motivo, el número de sesiones registradas no coincide exactamente con el número de intentos programados en el script. Esta pérdida afecta la representatividad del corpus si se lo interpretara como muestra estadística, pero no compromete la validación funcional del pipeline, ya que el experimento produjo eventos suficientes de los tipos requeridos para verificar las rutas principales del sistema: conexión, autenticación, ejecución de comandos, descarga de archivo, persistencia en PostgreSQL, indexación en Elasticsearch, visualización en Kibana y alertado por Telegram.

Las veintiséis alertas de Telegram recibidas durante el experimento —veinte alertas de login exitoso y seis alertas de descarga de archivo con resultado de VirusTotal— fueron verificadas

en el celular de los investigadores y confirmaron el correcto funcionamiento del subsistema de alertas.

4.7 Fase 7: Aprovisionamiento del VPS y exposición a internet real

4.7.1 Selección del proveedor y configuración del servidor

La decisión de usar DigitalOcean como proveedor de VPS para esta fase del proyecto respondió a varios criterios: la simplicidad de la interfaz de administración, que facilita el aprovisionamiento sin conocimientos avanzados de infraestructura cloud; la disponibilidad de un plan económico de seis dólares mensuales con recursos suficientes para ejecutar el honeypot; la presencia de centros de datos en la región de Amsterdam, que ofrece buena conectividad y exposición a tráfico proveniente de Europa; y la documentación abundante disponible para la configuración de Ubuntu en DigitalOcean. El servidor fue aprovisionado con Ubuntu 24.04 LTS —la versión más reciente con soporte de largo plazo en el momento del proyecto— con 1 GB de RAM, 25 GB de almacenamiento SSD y 1000 GB de transferencia mensual incluidos en el plan. La configuración de nombre de host honeypot-oscorp permite identificar el servidor en los registros del proveedor. La autenticación SSH para el acceso administrativo fue configurada inicialmente con contraseña, con la contraseña Oscorp2026Honeypot, y luego se añadió autenticación mediante clave pública RSA de 4096 bits para la sincronización automática de logs.

4.7.2 Despliegue de Cowrie en el VPS y primera captura de tráfico real

El proceso de despliegue de Cowrie en el VPS siguió exactamente la misma secuencia que en el entorno local, con dos diferencias significativas. La primera diferencia es que el contenedor Cowrie en el VPS mapea el puerto 2222 del contenedor al puerto 2222 del VPS, que al tener IP pública directa es accesible desde internet sin ninguna configuración adicional de firewall o NAT. La segunda diferencia es que el directorio de logs se monta en /opt/honeypot/logs en el sistema de archivos del VPS, lo que permite que el script de sincronización acceda a los logs mediante SCP. El resultado más notable de esta fase fue la velocidad con que el honeypot comenzó a recibir tráfico malicioso autónomo. En menos de treinta minutos desde la activación del contenedor Cowrie en el VPS, el primer atacante externo —IP 66.132.195.69— había intentado conectarse al servicio SSH emulado. Este resultado validó de forma contundente la hipótesis secundaria sobre tráfico real: internet está permanentemente escaneado por bots automatizados que buscan servicios SSH expuestos, y un servidor con IP pública en el puerto 2222 es encontrado y atacado en cuestión de minutos, sin ninguna acción de publicidad o atracción por parte de los investigadores.

4.7.3 Configuración de la sincronización automática de logs

La sincronización automática de logs entre el VPS y la PC local se implementó mediante un script PowerShell —sync_vps.ps1— que ejecuta periódicamente el comando SCP con la clave SSH configurada sin contraseña. El script corre en un bucle infinito con una pausa de cincuenta

y cinco segundos entre cada ejecución, lo que garantiza que el archivo `cowrie_vps.json` en la PC local se actualice aproximadamente cada minuto, en sincronía con el intervalo del Schedule Trigger de `n8n`. La configuración de la autenticación SSH sin contraseña entre la PC local y el VPS requirió generar un par de claves RSA de 4096 bits en la PC local mediante `ssh-keygen` y copiar la clave pública al archivo `authorized_keys` del

servidor VPS con los permisos correctos. Este mecanismo garantiza que el script de sincronización puede conectarse al VPS y descargar el archivo de logs sin intervención del usuario, incluso después de reinicios del sistema. Un problema técnico recurrente que se presentó con la sincronización fue el error «Broken pipe» que SCP reportaba ocasionalmente cuando intentaba sobrescribir el archivo `cowrie_vps.json` mientras Cowrie estaba escribiendo en él simultáneamente en el VPS. La solución fue asignar permisos explícitos de escritura para todos los usuarios sobre la carpeta de logs mediante el comando `icacls` con la bandera `/T`, que aplica los permisos de forma recursiva y permanente, incluyendo los archivos que se creen en el futuro.

4.8 Fase 8: Integración de VirusTotal y geolocalización de IPs

4.8.1 Proceso de integración de VirusTotal

La integración de VirusTotal en el workflow de `n8n` requirió la obtención de una API Key mediante el registro gratuito en la plataforma `virustotal.com`. El plan gratuito proporciona hasta quinientas consultas diarias y cuatro consultas por minuto, suficientes para el volumen de eventos de descarga de archivos que genera el honeypot en condiciones normales de operación. La API Key se configuró como una credencial de tipo Header Auth en `n8n`, con el nombre de encabezado `x-apikey`, lo que garantiza que la clave no queda expuesta en la definición del nodo HTTP Request del workflow. El endpoint de la API de VirusTotal utilizado es `GET https://www.virustotal.com/api/v3/files/{hash}`, donde `{hash}` se sustituye dinámicamente por el valor del campo `shasum` del evento de descarga, accedido mediante la expresión de `n8n` `$(Filtro Atacantes').item.json.shasum`. Es importante notar el uso de la referencia al nodo Filtro Atacantes en lugar del objeto `$json` estándar, que en este punto del flujo contiene los datos de la respuesta de `ip-api.com` y no los del evento original de Cowrie. Esta distinción técnica fue uno de los problemas más desafiantes del desarrollo y requirió comprender en profundidad el modelo de datos de `n8n` en flujos con múltiples transformaciones consecutivas. La respuesta de la API de VirusTotal para el hash `fb91d75a6bb430787a61b0aec5e374f580030f2878e1613eab5ca6310f7bbb9a` —el hash del archivo descargado desde `example.com` durante las pruebas— fue: `malicious: 0, undetected: 71, harmless: 5`. Este resultado —cero antivirus detectándolo como malware de setenta y seis analizados— era el esperado dado que el archivo descargado desde `example.com` es la respuesta estándar de ese servidor de prueba, no un binario malicioso real. Este resultado fue incorporado correctamente en la alerta de Telegram con el texto «Detectado por: 0/76 antivirus ↗No detectado como malware».

4.8.2 Proceso de integración de ip-api.com

La integración de ip-api.com fue técnicamente más simple que la de VirusTotal porque el servicio no requiere autenticación —la API es pública y no requiere API Key para uso no comercial dentro de los límites de rate—. El endpoint utilizado es GET `http://ip-api.com/json/{ip}`, donde {ip} se sustituye dinámicamente por el valor del campo `src_ip` del evento crítico, accedido mediante la expresión `$json.src_ip` dado que en este punto del flujo el contexto del nodo es el evento del Filtro Atacantes. La primera alerta completa con geolocalización recibida durante las pruebas incluyó los datos de la IP 201.190.175.63, que ip-api.com identificó como proveniente de Argentina, Mendoza, con ISP Arlink S.A. Este resultado era el esperado dado que esa IP corresponde a la conexión de prueba realizada por los propios investigadores desde su red doméstica de Mendoza. La confirmación de la correcta identificación geográfica validó el funcionamiento del subsistema de geolocalización. La alerta de mayor impacto analítico recibida durante el período de operación del VPS fue la correspondiente a la IP 47.251.178.17, que ip-api.com identificó como proveniente de United States, Minkler, con ISP Alibaba Cloud LLC. Esta alerta representó la primera evidencia de un atacante real de internet que había ingresado exitosamente al honeypot —con las credenciales `root/P`— y cuyo origen fue correctamente geolocalizando como infraestructura cloud de Alibaba en Estados Unidos, un origen frecuentemente asociado en la literatura de amenazas con bots de escaneo y fuerza bruta SSH.

Capítulo 5: Resultados y Análisis

5.1 Resultados del experimento controlado

5.1.1 Corpus de eventos generados

El experimento controlado de cuarenta sesiones produjo un corpus de eventos que constituye la base empírica principal de la validación del sistema. La siguiente tabla presenta el desglose de los doscientos diecinueve eventos registrados en PostgreSQL durante el experimento, clasificados por tipo según los valores del campo eventid:

Tipo de evento (eventid)	Descripción	Cantidad	% del total
cowrie.session.connect	Nueva conexión SSH establecida	26	11.9%
cowrie.login.success	Autenticación exitosa en la shell simulada	26	11.9%
cowrie.command.input — whoami	Comando de identificación del usuario	20	9.1%
cowrie.command.input — ls	Listado del directorio actual	16	7.3%
cowrie.command.input — cat /etc/passwd	Lectura del archivo de usuarios	14	6.4%
cowrie.command.input — wget malware.sh	Descarga de archivo malicioso simulado	16	7.3%
cowrie.command.input — uname -a	Información del sistema operativo	12	5.5%
cowrie.command.input — otros	Otros comandos ejecutados	23	10.5%
cowrie.session.file_download	Registro de archivo descargado con hash	6	2.7%
cowrie.login.failed	Intentos fallidos de autenticación (Hydra)	~60	~27.4%
cowrie.session.closed	Cierre de sesión SSH	26	11.9%

La distribución de comandos registrada en el experimento refleja el comportamiento típico de un atacante automatizado en la fase de reconocimiento post-autenticación: los comandos más frecuentes son los que proporcionan información sobre el sistema comprometido —whoami, uname, cat /etc/passwd— y los que permiten descargar herramientas adicionales —wget—.

Esta distribución es consistente con la observada en estudios académicos previos sobre el comportamiento de bots SSH, lo que sugiere que el script del experimento reproduce adecuadamente los patrones de ataque reales.

5.1.2 Métricas de desempeño del pipeline

Las métricas de desempeño del pipeline obtenidas durante el experimento controlado se presentan en la siguiente tabla. Los valores correspondientes al sistema automatizado derivan de la ejecución del experimento, de los registros almacenados en PostgreSQL, de las alertas efectivamente recibidas por Telegram y de la configuración operacional del Schedule Trigger de n8n. En cambio, los valores incluidos para el escenario manual no constituyen mediciones empíricas de un sistema alternativo en operación, sino estimaciones hipotéticas elaboradas por los autores con finalidad exclusivamente referencial. Por lo tanto, la tabla no debe interpretarse como una comparación experimental entre dos sistemas bajo iguales condiciones, sino como una forma de contextualizar funcionalmente qué aspectos del proceso manual busca superar la arquitectura automatizada desarrollada.

Métrica	Escenario manual hipotético	Sistema automatizado	Diferencia
Tiempo de procesamiento del evento	>5 minutos por evento	<60 segundos (máximo)	Reducción funcional estimada
MTTD promedio	>10 minutos (revisión periódica)	~30 segundos	Reducción teórica estimada
Intervención requerida por evento	Alta (revisión manual)	Ninguna (0%)	Intervención manual eliminada dentro del alcance del pipeline
Eventos procesados en el experimento	219 (manual, acumulados)	219 (automático, en tiempo real)	Igual cobertura
Alertas generadas automáticamente	0 (no existe sistema de alerta)	26 alertas completas	De 0 a 26
Enriquecimiento de los eventos	Ninguno	País + ciudad + ISP + VT	Nuevo: contexto completo
Trazabilidad por sesión	Baja (búsqueda manual en texto)	Alta (SQL por session ID)	Cualitativamente superior
Disponibilidad del análisis	Solo cuando el analista está activo	24/7 automatizado	Mayor continuidad operativa dentro del alcance del sistema

Nota. Los valores del escenario manual corresponden a estimaciones hipotéticas de los autores y no a mediciones empíricas de ningún sistema manual en operación. Su función es referencial e ilustrativa. En consecuencia, las diferencias expresadas en la tabla no deben interpretarse como resultados estadísticamente demostrados, sino como una comparación funcional orientada a mostrar el tipo de mejora operativa que la automatización permite alcanzar dentro del alcance del proyecto.

5.1.3 Análisis de las alertas de Telegram recibidas

Las veintiséis alertas de Telegram recibidas durante el experimento controlado representan el resultado más directamente verificable del sistema desde la perspectiva del operador: son la evidencia de que el pipeline funciona de extremo a extremo, desde la captura del evento por Cowrie hasta la notificación en el dispositivo del operador. Las alertas fueron recibidas en dos formatos distintos según el tipo de evento: Las alertas de tipo login exitoso —veinte en total— incluyeron: el tipo de evento, la IP atacante con su geolocalización completa (país, ciudad, ISP), el nombre de usuario y la contraseña utilizados en la autenticación, el identificador de sesión, la fecha y la hora del evento en zona horaria Argentina, y la firma del sistema.

Las alertas de tipo descarga de archivo —seis en total— incluyeron adicionalmente el hash SHA-256 del archivo descargado y el resultado de la consulta a VirusTotal expresado como número de antivirus que detectaron el archivo como malware sobre el total de motores analizados. La primera alerta fue recibida aproximadamente treinta segundos después del inicio del experimento, confirmando el MTTD estimado. Las alertas posteriores continuaron llegando a intervalos de aproximadamente sesenta segundos a medida que el Schedule Trigger de n8n procesaba los eventos acumulados. Todas las alertas fueron verificadas y resultaron correctas en todos sus campos, sin ningún caso de alerta con datos incorrectos o formato malformado.

5.2 Resultados de la exposición a internet real

5.2.1 Caracterización del tráfico real capturado

IP atacante	País	Ciudad	ISP	Eventos generados	Tipo de actividad
47.251.178.17	Estados Unidos	Minkler	Alibaba Cloud LLC	Login exitoso: root/P	Fuerza bruta + login
20.163.14.227	Estados Unidos	—	Microsoft Azure	25 eventos de conexión	Escaneo y reconocimiento
66.132.195.69	Estados Unidos	—	EEUU (ISP no identificado)	6 eventos de exploración	Conexión y exploración
45.142.193.6	Europa	—	Infraestructura VPS	2 eventos de conexión	Intento de conexión
201.190.175.63	Argentina	Mendoza	Arlink S.A.	Múltiples (investigadores)	Pruebas controladas

Durante las primeras veinticuatro horas de operación del VPS con IP pública, el honeypot capturó más de cuatrocientos eventos provenientes de múltiples IPs externas. Este resultado confirma de manera contundente la hipótesis sobre tráfico real: internet está permanentemente escaneado por bots automatizados, y un servicio SSH expuesto en el puerto 2222 con IP pública directa es encontrado y atacado en cuestión de minutos sin ninguna intervención activa de los

investigadores. Las IPs externas más relevantes capturadas durante el período inicial de operación, junto con su geolocalización obtenida mediante ip-api.com, se presentan en la siguiente tabla:

La presencia de IPs de infraestructura cloud de Microsoft Azure (20.163.14.227) y Alibaba Cloud LLC (47.251.178.17) entre los atacantes reales capturados es especialmente significativa desde el punto de vista de la inteligencia de amenazas: estas IPs corresponden a máquinas virtuales alquiladas en plataformas cloud que son utilizadas frecuentemente por actores maliciosos para realizar ataques de fuerza bruta SSH, precisamente porque las IPs de los principales proveedores cloud tienen menor probabilidad de estar incluidas en listas de bloqueo que las IPs de ISPs residenciales o de proveedores de hosting más pequeños.

5.2.2 Funcionamiento del enriquecimiento con tráfico real

La prueba más significativa del subsistema de enriquecimiento con tráfico real fue la alerta recibida para la IP 47.251.178.17, que corresponde al primer atacante externo que logró un login exitoso en el honeypot del VPS. La alerta completa recibida en Telegram incluyó: tipo LOGIN EXITOSO, IP atacante 47.251.178.17, origen United States, Minkler, ISP Alibaba Cloud LLC, usuario root, contraseña P, identificador de sesión d7a9a1775046, fecha 21/4/2026 y hora 10:56:14 en zona horaria Argentina. Esta alerta demuestra que el sistema enriquece correctamente los eventos de tráfico real con la misma calidad que los eventos del experimento controlado. La alerta por el login exitoso de un atacante externo real es, desde el punto de vista académico y operativo, el resultado más significativo del proyecto: demuestra que el sistema no solo funciona en condiciones controladas de laboratorio, sino también con tráfico malicioso real proveniente de actores externos que no conocen ni controlan el entorno. La geolocalización correcta del atacante como proveniente de infraestructura de Alibaba Cloud en Estados Unidos y la ausencia de alertas falsas positivas durante el período de observación validan tanto la exactitud del subsistema de enriquecimiento como la efectividad del Filtro Atacantes para distinguir eventos críticos de eventos de menor relevancia.

5.3 Análisis de trazabilidad y coherencia del sistema

La trazabilidad del evento a través de todas las capas del sistema fue verificada para múltiples eventos del experimento controlado, utilizando el campo session como identificador de trazabilidad. El procedimiento de verificación consistió en tomar el session ID de una alerta de Telegram, buscar todos los eventos con ese session ID en PostgreSQL mediante una consulta SQL, verificar que los mismos eventos estuvieran disponibles en Elasticsearch, y confirmar que el pico de actividad correspondiente al momento del evento fuera visible en el histograma temporal de Kibana. Esta verificación multimodal —Telegram, PostgreSQL, Elasticsearch, Kibana— confirma que el mismo evento está correctamente representado en todos los componentes del sistema y que puede ser rastreado de forma consistente desde cualquier punto de entrada al análisis. La coherencia entre las cuatro representaciones del evento valida la

integridad del pipeline y demuestra que no existen pérdidas de datos ni inconsistencias entre las capas del sistema.

5.4 Límites de inferencia de los resultados

La interpretación de los resultados presentados en este capítulo debe realizarse dentro de los límites metodológicos explícitamente reconocidos. El valor de MTTD de treinta segundos es una estimación basada en el diseño del sistema, no una medición directa con instrumentación precisa. El experimento de

cuarenta sesiones es un experimento único realizado en condiciones específicas, no una serie de experimentos replicados para calcular variabilidad estadística. El tráfico real capturado durante las primeras veinticuatro horas de operación del VPS representa una muestra no representativa del comportamiento global de los atacantes en internet, aunque sí es suficientemente indicativa para confirmar la hipótesis de que el tráfico malicioso autónomo se presenta en un plazo corto. Estas limitaciones no invalidan los resultados sino que los contextualizan correctamente: el sistema funciona como fue diseñado, dentro de los parámetros que su diseño establece, y los resultados obtenidos son consistentes con las hipótesis planteadas. Una validación más rigurosa —con medición exacta del MTTD, múltiples experimentos replicados y análisis estadístico de la variabilidad— constituye una línea natural de trabajo futuro que se recomienda en el capítulo de recomendaciones.

Asimismo, debe considerarse que el experimento controlado de cuarenta sesiones fue diseñado con finalidad de validación funcional y no como muestra estadística representativa del comportamiento general de atacantes SSH. El hecho de que Cowrie haya consolidado veintiséis sesiones únicas sobre las cuarenta interacciones programadas constituye una limitación del corpus experimental, aunque no impide validar que el pipeline procesa correctamente los tipos de evento necesarios para cumplir los objetivos técnicos del proyecto.

Capítulo 6: Discusión

6.1 Contrastación de las hipótesis con los resultados

El análisis de los resultados presentados en el capítulo anterior permite contrastar las cuatro hipótesis de la investigación con la evidencia empírica obtenida. Esta contrastación constituye el núcleo argumental del capítulo de discusión y permite situar los resultados en el marco más amplio de la investigación en ciberseguridad y educación técnica.

La hipótesis principal —que la integración del honeypot con el pipeline automatizado permite mejorar funcionalmente la gestión de eventos de intrusión SSH dentro del alcance definido— encuentra respaldo funcional en los resultados del experimento controlado. El sistema permitió procesar y alertar eventos dentro del intervalo operativo previsto por el flujo automatizado, con un MTDD promedio estimado de aproximadamente treinta segundos. No obstante, esta mejora debe interpretarse como una reducción teórica de latencia operativa frente a una revisión manual periódica de referencia, y no como una mejora porcentual estadísticamente demostrada mediante grupo de comparación. La eliminación completa de la intervención humana en el procesamiento de eventos —tasa de automatización del cien por ciento— es igualmente significativa: no solo reduce la carga de trabajo del operador sino que elimina el riesgo de error humano en el proceso de procesamiento y alertado. El enriquecimiento automático con geolocalización y resultado de VirusTotal añade una dimensión de inteligencia que no existe en ningún escenario de revisión manual básica. Sin embargo, la contrastación de la hipótesis principal requiere una cualificación importante. La mejora demostrada es funcional, no estadística: el experimento muestra que el sistema produce los resultados esperados en las condiciones del experimento, no que produce una mejora estadísticamente significativa sobre el escenario manual en un sentido formal. Una contrastación estadística rigurosa requeriría medir con precisión el tiempo de detección en ambos escenarios para un conjunto de eventos controlados y aplicar un test de hipótesis sobre la diferencia, lo que está fuera del alcance de este trabajo pero constituye una línea de investigación futura natural.

La segunda hipótesis, referida a la reproducibilidad potencial del laboratorio, encuentra respaldo documental pero no validación empírica externa. El trabajo produce anexos técnicos, scripts, capturas, configuraciones y descripciones de fases que favorecen la posibilidad de que otro equipo académico reconstruya el sistema. Sin embargo, dado que no se realizó una replicación independiente por parte de terceros, esta hipótesis no puede considerarse empíricamente validada en sentido estricto. Su alcance debe interpretarse como reproducibilidad potencial documentada, no como reproducibilidad demostrada.

La tercera hipótesis —sobre la riqueza analítica de los logs de Cowrie— fue validada directamente por el análisis de los doscientos diecinueve eventos del experimento: los logs contienen información suficientemente rica y estructurada —con campos de session, eventid, timestamp, src_ip, username, password, command y shasum— para identificar

comportamientos relevantes, reconstruir sesiones completas y alimentar el subsistema de inteligencia de amenazas de forma automatizada.

La cuarta hipótesis —sobre el tráfico real— fue la que obtuvo la validación más contundente e inmediata: el honeypot del VPS comenzó a recibir ataques de bots externos en menos de treinta minutos desde su activación, confirmando que internet está permanentemente escaneado y que un servicio SSH con IP pública es encontrado y atacado de forma autónoma en cuestión de minutos.

6.2 Diferenciación del proyecto frente a enfoques tecnológicos similares

La contribución del proyecto OSCOP puede comprenderse con mayor precisión si se lo compara no con trabajos locales específicos, sino con enfoques tecnológicos habituales en laboratorios académicos de ciberseguridad y monitoreo de intrusiones. En este tipo de entornos, es frecuente que las herramientas se utilicen de manera parcial o fragmentada: un honeypot puede desplegarse para observar intentos de acceso, un stack de visualización puede utilizarse para explorar registros, o una herramienta de automatización puede emplearse para ejecutar tareas aisladas. Sin embargo, el valor diferencial de OSCOP reside en la integración funcional de estas capacidades dentro de una arquitectura única, continua y documentada.

Desde esta perspectiva, el proyecto se diferencia en cuatro dimensiones principales. La primera es la exposición controlada a internet real mediante un servidor VPS con IP pública, lo que permitió complementar las pruebas generadas en laboratorio con eventos provenientes de bots externos que encontraron el honeypot de forma autónoma. Esta dimensión amplía el valor empírico del sistema, porque permite observar comportamiento malicioso no producido por los propios investigadores.

La segunda dimensión diferencial es el enriquecimiento automático con inteligencia de amenazas. La integración con VirusTotal permite asociar los hashes SHA-256 de archivos descargados por los atacantes con resultados de reputación externa, transformando un dato técnico aislado en un indicador de compromiso con mayor valor analítico. De manera complementaria, la integración con ip-api.com permite incorporar país, ciudad e ISP de origen de las IPs atacantes, aportando contexto geográfico y operacional a las alertas generadas.

La tercera dimensión es la articulación entre almacenamiento estructurado, indexación analítica y visualización interactiva. PostgreSQL preserva la trazabilidad relacional de los eventos, Elasticsearch permite búsquedas y agregaciones de alto rendimiento, y Kibana ofrece una capa visual que facilita la interpretación de patrones temporales, comandos ejecutados, IPs recurrentes y actividad asociada a sesiones de ataque.

La cuarta dimensión es la documentación integral del proceso de implementación. El trabajo no se limita a presentar el resultado final del sistema, sino que registra fases de construcción, decisiones de diseño, errores encontrados, ajustes técnicos, scripts utilizados, configuración de servicios y evidencias de funcionamiento. Esta documentación fortalece el valor académico del proyecto, porque permite que el laboratorio sea comprendido, auditado y potencialmente reproducido por otros equipos con conocimientos técnicos equivalentes.

Por lo tanto, la originalidad práctica de OSCOP no debe ubicarse en la creación aislada de un honeypot, de un dashboard o de una automatización puntual, sino en la integración verificable de captura, procesamiento, almacenamiento, indexación, visualización, enriquecimiento y alertado dentro de un pipeline open source de bajo costo, validado funcionalmente tanto con tráfico controlado como con tráfico real de internet.

6.3 Implicaciones pedagógicas del laboratorio desarrollado

El laboratorio desarrollado tiene implicaciones pedagógicas que van más allá del valor académico de la tesis como trabajo final. Como recurso formativo, el sistema permite que los estudiantes de ciberseguridad experimenten en tiempo real el ciclo completo del monitoreo de amenazas: desde la observación de un ataque en el honeypot hasta la recepción de la alerta enriquecida en el celular, pasando por el análisis en Kibana y la consulta en PostgreSQL. Esta experiencia integral —que ningún libro de texto o simulación académica puede reproducir con la misma fidelidad— tiene el potencial de generar la comprensión profunda que distingue a un técnico competente de alguien que solo conoce la teoría. Zhao et al. (2023) argumentan que los marcos educativos sustentados en herramientas abiertas son especialmente valiosos porque combinan accesibilidad económica, relevancia tecnológica y cercanía con las prácticas profesionales reales. El laboratorio OSCOP cumple estos tres criterios: es

completamente gratuito en su componente de software, usa las mismas herramientas que se usan en entornos de operaciones de seguridad profesionales, y reproduce —en escala reducida pero con fidelidad conceptual— el tipo de trabajo que realiza un analista de un SOC real. La dimensión más valiosa pedagógicamente es probablemente la exposición a los problemas técnicos reales que surgen durante la implementación: el desajuste de zona horaria en PostgreSQL, el conflicto de permisos en el archivo de sincronización, el problema de acceso a datos de nodos anteriores en n8n cuando el contexto del nodo actual corresponde a los datos transformados por una API intermedia, la saturación de memoria en Elasticsearch en servidores con recursos limitados. Ninguno de estos problemas está en los tutoriales ni en la documentación oficial, y resolverlos requiere habilidades de diagnóstico, búsqueda de información y resolución de problemas que son exactamente las que se desarrollan en la práctica profesional real.

6.4 Posicionamiento del sistema en el ecosistema de herramientas de seguridad

Es importante situar el sistema desarrollado con honestidad intelectual dentro del ecosistema más amplio de herramientas y plataformas de seguridad, reconociendo tanto sus fortalezas como sus limitaciones en términos comparativos. El sistema OSCOP no es, ni pretende ser, un sustituto de las plataformas SOAR comerciales de primera línea ni de los sistemas SIEM maduros que se utilizan en centros de operaciones de seguridad corporativos. Las plataformas SOAR comerciales como Splunk SOAR, IBM QRadar SOAR o Palo Alto XSOAR ofrecen capacidades que el sistema OSCOP no implementa: centenares de integraciones preconfiguradas con herramientas de seguridad, respuesta activa automática con playbooks complejos, análisis de comportamiento basado en machine learning, gestión de casos de incidentes, correlación estadística de eventos entre múltiples fuentes, y soporte empresarial.

Lo que el sistema OSCOP ofrece es algo conceptualmente distinto pero complementario: una implementación académica accesible, completamente transparente en su funcionamiento, que demuestra los principios fundamentales del paradigma SOAR con herramientas gratuitas, y que puede ser comprendida, modificada y extendida por cualquier estudiante de programación sin barreras de costo ni de caja negra propietaria.

Capítulo 7: Conclusiones

La presente investigación partió de un problema concreto y bien delimitado: la inexistencia, en el contexto académico de la Tecnicatura Universitaria en Programación de la UTN Facultad Regional Mendoza, de un laboratorio automatizado de honeypots que integrara captura, procesamiento, enriquecimiento, almacenamiento, indexación, visualización y alertado de eventos de intrusión en un pipeline coherente, documentado, reproducible y de bajo costo. A lo largo de nueve fases de desarrollo incremental, la investigación diseñó, implementó y validó funcionalmente un sistema que resuelve ese problema de manera técnicamente sólida y académicamente significativa.

La primera conclusión, y la más fundamental, es que la integración es posible y que su resultado supera cualitativamente la suma de sus partes. Cowrie, n8n, PostgreSQL, Elasticsearch, Kibana, VirusTotal, ip-api.com y Telegram son herramientas individualmente valiosas, pero su valor se multiplica cuando se las integra en un pipeline coherente que transforma el dato bruto del sensor en alertas enriquecidas con inteligencia de amenazas. Esta multiplicación de valor a través de la integración es, en sí misma, una lección técnica de primer orden que el laboratorio materializa de forma concreta y observable.

La segunda conclusión es que la automatización del pipeline de monitoreo es una mejora cualitativa, no meramente cuantitativa, respecto de la revisión manual de logs. La diferencia entre revisar periódicamente un archivo de texto y recibir una alerta enriquecida en el celular en treinta segundos no es solo una diferencia de velocidad: es una diferencia de paradigma. El primer modelo exige que el analista esté activo y presente para detectar amenazas; el segundo modelo permite que el analista sea notificado en tiempo real independientemente de lo que esté haciendo, y que la notificación incluya ya el contexto analítico necesario para una primera evaluación de criticidad. Esta diferencia es la que separa la ciberseguridad reactiva de la ciberseguridad proactiva.

La tercera conclusión se refiere al VPS y a la exposición a internet real: la transición desde el laboratorio local hacia el servidor con IP pública no es un paso opcional o cosmético del proyecto, sino una validación esencial de que el sistema funciona con condiciones de operación genuinas. El hecho de que el honeypot comenzara a recibir ataques reales en menos de treinta minutos desde su activación —sin ninguna acción de atracción por parte de los investigadores— demuestra empíricamente que los bots de escaneo SSH son una amenaza constante y omnipresente en internet, y que un sistema de monitoreo que no sea capaz de operar bajo estas condiciones reales tiene un valor académico y práctico limitado.

La cuarta conclusión concierne al enriquecimiento con inteligencia de amenazas: la integración de VirusTotal e ip-api.com eleva el sistema desde la categoría de honeypot con logging —que es donde la mayoría de los proyectos académicos similares se detienen— hacia la categoría de plataforma de inteligencia de amenazas SOAR-lite. Esta distinción no es solo terminológica: implica que el sistema produce, de forma automática y sin intervención humana, información

de inteligencia operativamente útil que no existiría sin la integración de estas APIs. El analista que recibe una alerta del sistema OSCORP no recibe solo una notificación de que algo ocurrió; recibe un informe mini que le dice qué ocurrió, dónde geográficamente se originó y si el archivo involucrado es reconocido como malware por la comunidad global de antivirus.

La quinta conclusión es de naturaleza económica y social: el sistema demuestra que es posible construir una plataforma de inteligencia de amenazas operativamente comparable en sus funciones centrales con soluciones comerciales SOAR de primer nivel, utilizando exclusivamente herramientas gratuitas y servicios de API pública, con un costo de operación de seis dólares mensuales. Esta conclusión tiene implicaciones que trascienden el ámbito académico: si una tecnicatura universitaria puede construir y validar este tipo de sistema como trabajo final de carrera, entonces municipios, cooperativas, pequeñas empresas, hospitales y otras organizaciones de recursos moderados pueden adoptar modelos similares para mejorar su postura de seguridad sin incurrir en costos prohibitivos.

La sexta y última conclusión es de naturaleza pedagógica: el laboratorio desarrollado tiene el potencial de convertirse en una plataforma formativa de excepcional valor para la enseñanza de ciberseguridad en carreras de programación y sistemas. Reúne en un único artefacto técnico — accesible, documentado y reproducible— los conceptos centrales de múltiples disciplinas: honeypots y engaño defensivo, automatización de flujos con n8n, bases de datos relacionales con PostgreSQL, indexación y búsqueda con Elasticsearch, visualización analítica con Kibana, APIs REST y servicios cloud, contenedorización con Docker, e infraestructura cloud con VPS. Ningún libro de texto puede proporcionar la comprensión integrada y encarnada que la experiencia directa de construir, operar y analizar este sistema puede generar.

No obstante, como todo desarrollo tecnológico validado en un alcance académico delimitado, el sistema OSCORP abre líneas de mejora que pueden orientar futuras iteraciones del laboratorio. Estas líneas no invalidan los resultados obtenidos, sino que derivan precisamente de las limitaciones reconocidas durante la investigación.

7.1 Trabajos futuros

A partir de las limitaciones reconocidas durante el desarrollo de la investigación y de los resultados obtenidos en la validación funcional del sistema, se identifican varias líneas de trabajo futuro que permitirían ampliar, fortalecer y profundizar el laboratorio OSCORP

La primera línea de trabajo futuro consiste en instrumentar una medición directa y precisa del tiempo medio de detección. En la versión actual del sistema, el MTDD se estimó a partir del intervalo de ejecución del Schedule Trigger de n8n, por lo que representa una aproximación funcional y no una medición evento por evento. Una mejora metodológica relevante consistiría en registrar, para cada evento crítico, el timestamp exacto de ocurrencia en Cowrie, el momento de procesamiento en n8n y el momento efectivo de envío de la alerta por Telegram. Esta instrumentación permitiría calcular el MTDD real para cada evento, obtener promedios, valores

máximos, mínimos y desviaciones, y comparar el desempeño del pipeline bajo diferentes configuraciones de frecuencia de ejecución.

La segunda línea consiste en replicar el experimento controlado con un mayor número de sesiones, múltiples ejecuciones y condiciones variables de carga. El experimento de cuarenta sesiones permitió validar funcionalmente el pipeline, pero no fue diseñado como una muestra estadística representativa del comportamiento general de atacantes SSH. En una versión futura, podrían ejecutarse campañas repetidas en distintos momentos, con diferentes herramientas de ataque, intervalos de conexión, listas de credenciales y secuencias de comandos. Esto permitiría analizar la estabilidad del sistema, la consistencia del registro de sesiones, la tasa de pérdida entre intentos programados y sesiones consolidadas, y el comportamiento del pipeline bajo volúmenes más altos de eventos.

La tercera línea de trabajo futuro corresponde a la ampliación del laboratorio hacia otros protocolos y servicios expuestos. La presente investigación se concentró exclusivamente en eventos SSH capturados mediante Cowrie, lo cual permitió delimitar adecuadamente el alcance técnico del proyecto. Sin embargo, un laboratorio de monitoreo más completo podría incorporar honeypots especializados para HTTP, FTP, SMB, RDP u otros servicios frecuentemente atacados. Esta extensión permitiría comparar patrones de ataque entre protocolos, enriquecer el corpus de eventos y acercar el laboratorio a una arquitectura de observación de amenazas más representativa de entornos organizacionales reales.

La cuarta línea consiste en validar externamente la reproducibilidad del sistema. Si bien el proyecto fue documentado con un nivel de detalle orientado a permitir su reproducción, la hipótesis de reproducibilidad no fue contrastada mediante la implementación independiente por parte de otro equipo. En futuras iteraciones, podría solicitarse a estudiantes o docentes externos al proyecto que repliquen el laboratorio a partir de la documentación técnica, registrando dificultades, ambigüedades, tiempos de instalación, errores frecuentes y ajustes necesarios. Esta validación permitiría fortalecer la calidad pedagógica de los anexos y convertir el proyecto en una guía de laboratorio reutilizable.

La quinta línea de mejora se vincula con la visualización geográfica y analítica de los eventos. La integración con ip-api.com permitió enriquecer las alertas con país, ciudad e ISP de origen, pero una versión futura podría llevar esa información a dashboards geográficos dentro de Kibana. Esto permitiría representar en mapas la distribución de IPs atacantes, identificar concentraciones por región, analizar proveedores de infraestructura recurrentes y observar la evolución temporal de los orígenes de ataque. Esta mejora aumentaría el valor didáctico y analítico del sistema, especialmente para actividades de enseñanza vinculadas con inteligencia de amenazas.

La sexta línea de trabajo futuro corresponde al endurecimiento operativo del pipeline. En la versión actual, el procesamiento depende de la disponibilidad de la PC local donde se ejecutan n8n, PostgreSQL, Elasticsearch y Kibana. Una evolución natural del proyecto sería migrar estos componentes a una infraestructura persistente, ya sea en el propio VPS, en una segunda

instancia cloud o en un servidor institucional. Esta modificación permitiría reducir la latencia asociada a períodos en los que la PC local permanece apagada, mejorar la continuidad operativa del sistema y acercar la arquitectura a condiciones de despliegue más estables.

Finalmente, una línea futura de mayor complejidad consistiría en incorporar mecanismos de correlación avanzada y respuesta automatizada controlada. Sin abandonar el enfoque ético y pasivo del honeypot, podrían agregarse reglas de correlación que identifiquen IPs recurrentes, comandos especialmente riesgosos, secuencias de comportamiento post-autenticación o descargas repetidas de archivos con hashes maliciosos. Estas reglas podrían generar niveles diferenciados de criticidad en las alertas y alimentar reportes periódicos de inteligencia de amenazas. Cualquier mecanismo de respuesta activa, como bloqueo automático de IPs, debería ser evaluado cuidadosamente desde el punto de vista legal, ético y académico antes de su implementación.

Referencias Bibliográficas

Cowrie Project. (2024). Cowrie SSH and Telnet Honeypot: Official documentation and GitHub repository. <https://cowrie.readthedocs.io/>

DigitalOcean. (2024). Droplets documentation: Virtual machines in the cloud. <https://docs.digitalocean.com/products/droplets/>

Docker, Inc. (2024). Docker documentation: The complete guide to containerization. <https://docs.docker.com/>

Elastic N.V. (2024). Elasticsearch reference documentation (Version 8.x). <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>

Elastic N.V. (2024). Kibana documentation: Visualization and dashboards. <https://www.elastic.co/guide/en/kibana/current/index.html>

ip-api.com. (2024). IP Geolocation API documentation. <https://ip-api.com/docs/>

MITRE Corporation. (2024). MITRE ATT&CK Framework: A knowledge base of adversary tactics and techniques. <https://attack.mitre.org/>

n8n GmbH. (2024). n8n documentation: Workflow automation for technical people. <https://docs.n8n.io/>

National Institute of Standards and Technology. (2006). Guide to Computer Security Log Management (NIST Special Publication 800-92). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-92>

National Institute of Standards and Technology. (2012). Computer Security Incident Handling Guide (NIST Special Publication 800-61, Revision 2). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-61r2>

National Institute of Standards and Technology. (2024). The NIST Cybersecurity Framework (CSF) 2.0 (NIST Cybersecurity White Paper No. 29). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.CSWP.29>

Omar, M., & Abed, A. (2023). Honeypot-based intrusion detection: A comprehensive survey of techniques and applications. Journal of Information Security and Applications, 78, 103601. <https://doi.org/10.1016/j.jisa.2023.103601>

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>

Spitzner, L. (2002). Honeypots: Tracking hackers. Addison-Wesley Professional.

van Hauser, T. (2024). THC Hydra: A very fast network logon cracker. GitHub. <https://github.com/vanhauser-thc/thc-hydra>

VirusTotal. (2024). VirusTotal API v3 reference documentation. <https://developers.virustotal.com/reference/overview>

Zhang, Y., Chen, X., & Liu, Q. (2022). Threat intelligence extraction from honeypot logs using machine learning and natural language processing. *IEEE Transactions on Network and Service Management*, 19(3), 2891–2904. <https://doi.org/10.1109/TNSM.2022.3151890>

Zhao, J., Yan, Q., Li, J., Shao, M., He, Z., & Li, B. (2023). Cybersecurity education in universities: A systematic literature review. *Computers & Education*, 196, 104731. <https://doi.org/10.1016/j.compedu.2023.104731>

Anexo Tecnico

Anexo A — Arquitectura del sistema

Tabla de todos los componentes del sistema con su función, tecnología, ubicación y forma de acceso.

Componente	Tecnología	Ubicación	Acceso
Honeypot SSH (producción)	Cowrie / Docker	VPS Amsterdam (167.71.8.253)	Puerto 2222 — IP pública
Honeypot SSH (laboratorio)	Cowrie / Docker	PC local (contenedor friendly_kalam)	Puerto 2222 — red local
Sincronización de logs	Script PowerShell + SCP	PC local (automático cada 55s)	Clave SSH RSA 4096 bits
Motor de automatización	n8n / Docker	PC local (contenedor n8n)	http://localhost:5678
Base de datos relacional	PostgreSQL / Docker	PC local (contenedor db-honeypot)	Puerto 5433
Motor de indexación	Elasticsearch / Docker	PC local (contenedor elasticsearch)	Puerto 9200
Visualización analítica	Kibana / Docker	PC local (contenedor kibana)	http://localhost:5601
Análisis de malware	API VirusTotal v3	Cloud externo (gratuito)	HTTPS — x-apikey header
Geolocalización de IPs	API ip-api.com	Cloud externo (gratuito)	HTTP REST — sin autenticación
Notificaciones en tiempo real	Bot de Telegram	Dispositivo móvil	Bot API de Telegram
VM de ataques controlados	Ubuntu / VirtualBox	PC local	Red interna — puerto 2222

Anexo B — Esquema de base de datos PostgreSQL

Script SQL de creación de la tabla principal del sistema. Se ejecuta una sola vez al iniciar el proyecto para definir la estructura de persistencia de eventos.

```
-- Creación de la base de datos
```

```
CREATE DATABASE honeypot;
```

```
-- Conexión a la base de datos
```

```
\c honeypot
```

```
-- Tabla principal de eventos capturados por Cowrie
```

```
CREATE TABLE eventos (  
    id          SERIAL PRIMARY KEY,  
    timestamp   TIMESTAMPTZ,  
    src_ip      VARCHAR(45),  
    username    VARCHAR(255),  
    password    VARCHAR(255),  
    command     TEXT,  
    shasum      VARCHAR(64),  
    session     VARCHAR(50),  
    eventid     VARCHAR(100),  
    url         TEXT,  
    hash        TEXT  
);
```

```
-- Índice para consultas frecuentes por sesión
```

```
CREATE INDEX idx_session ON eventos(session);
```

```
-- Índice para consultas por IP de origen
```

```
CREATE INDEX idx_src_ip ON eventos(src_ip);
```

```
-- Índice para consultas por tipo de evento
```

```
CREATE INDEX idx_eventid ON eventos(eventid);
```


Descripción de los campos principales de la tabla:

Campo	Tipo	Descripción
id	SERIAL PRIMARY KEY	Identificador autoincremental único por evento
timestamp	TIMESTAMPZ	Marca de tiempo del evento con zona horaria (UTC)
src_ip	VARCHAR(45)	Dirección IP de origen del atacante (soporta IPv4 e IPv6)
username	VARCHAR(255)	Nombre de usuario utilizado en el intento de autenticación
password	VARCHAR(255)	Contraseña utilizada en el intento de autenticación
command	TEXT	Comando ejecutado dentro de la shell simulada de Cowrie
shasum	VARCHAR(64)	Hash SHA-256 del archivo descargado por el atacante
session	VARCHAR(50)	Identificador único de sesión SSH asignado por Cowrie
eventid	VARCHAR(100)	Tipo de evento de Cowrie (cowrie.login.success, etc.)
url	TEXT	URL desde la que se descargó un archivo malicioso
hash	TEXT	Hash de integridad del evento completo

Consulta de verificación para ver los últimos 15 eventos registrados:

```
SELECT timestamp, src_ip, username, password, session, eventid
FROM eventos
ORDER BY timestamp DESC
LIMIT 15;
```

Anexo C — Código del nodo Parser NDJSON + Timestamp

Código JavaScript completo del nodo Code de n8n que implementa el parseo del archivo de logs NDJSON y el control de timestamps para evitar el reprocesamiento de eventos ya procesados. Este nodo es el componente de mayor complejidad técnica del workflow.

```
const binaryData = await this.helpers.getBinaryDataBuffer(0, 'data');
const rawText = binaryData.toString('utf8');
const lines = rawText.split('\n');

const lastTimestamp = $getWorkflowStaticData('global').lastTimestamp || '1970-01-01T00:00:00.000Z';

let newItems = [];
let maxTimestamp = lastTimestamp;

for (let line of lines) {
  if (line.trim() !== "") {
    try {
      let parsedJson = JSON.parse(line);
      if (parsedJson.timestamp && parsedJson.timestamp > lastTimestamp) {
        newItems.push({ json: parsedJson });
        if (parsedJson.timestamp > maxTimestamp) {
          maxTimestamp = parsedJson.timestamp;
        }
      }
    } catch (e) {}
  }
}

$getWorkflowStaticData('global').lastTimestamp = maxTimestamp;

return newItems;
```

Anexo D — Script de sincronización VPS (sync_vps.ps1)

Script PowerShell que descarga automáticamente el archivo de logs del VPS de DigitalOcean hacia la PC local cada 55 segundos mediante SCP con autenticación por clave SSH. Debe ejecutarse en una ventana de PowerShell aparte mientras el sistema está en operación.

```
# sync_vps.ps1 - Sincronización automática de logs del VPS
# Ejecutar en PowerShell: powershell -ExecutionPolicy Bypass -File
sync_vps.ps1
```

```
while ($true) {
    scp -i C:\Users\Elizabeth\.ssh\id_rsa_vps `
        root@167.71.8.253:/opt/honeypot/logs/cowrie.json `
        C:\Proyecto-honeypots\logs\cowrie_vps.json
    Write-Host "$(Get-Date) - cowrie_vps.json sincronizado desde
VPS"
    Start-Sleep -Seconds 55
}
```

Para aplicar permisos permanentes sobre la carpeta de logs (necesario para evitar errores de acceso):

```
icaccls "C:\Proyecto-honeypots\logs" /grant Todos:F /T
```

Para levantar el sistema completo desde cero, ejecutar en PowerShell:

```
# Paso 1 - Iniciar todos los contenedores Docker
docker start friendly_kalam db-honeypot n8n elasticsearch kibana
# Paso 2 - Verificar que estén activos
docker ps
# Paso 3 - Ejecutar el script de sincronización (en ventana aparte)
powershell -ExecutionPolicy Bypass -File C:\Proyecto-
honeypots\scripts\sync_vps.ps1
```

Anexo E — Script del experimento de 40 sesiones (experimento_40.sh)

Script bash ejecutado desde la VM Ubuntu en VirtualBox para el experimento controlado de cuarenta sesiones de ataque. Combina Hydra para fuerza bruta de credenciales (Fase 1) y sshpass para sesiones completas con comandos de reconocimiento (Fase 2).

```
#!/bin/bash
# experimento_40.sh - Experimento controlado de 40 sesiones SSH
# Ejecutar desde VM Ubuntu: bash /home/elizabeth/experimento_40.sh
```

```
IP="192.168.0.13" # IP del honeypot local (ajustar según ipconfig)
PORT="2222"
LOG="/home/elizabeth/experimento_log.txt"
```

```
echo "=== INICIO EXPERIMENTO 40 SESIONES ===" > $LOG
echo "Fecha: $(date)" >> $LOG
```

```
USERS=("root" "admin" "ubuntu" "pi" "user" "test" "oracle" "postgres")
PASSES=("admin" "123456" "password" "root" "1234" "qwerty" "toor" "test123")
COMMANDS=(
    "whoami; ls; exit"
    "uname -a; cat /etc/passwd; exit"
    "whoami; wget http://example.com/malware.sh; exit"
    "ls /etc; cat /etc/shadow; exit"
    "id; ps aux; exit"
)
```

```
echo "--- FASE 1: Fuerza bruta con Hydra (20 intentos) ---" >> $LOG
hydra -l root -P /home/elizabeth/passwords.txt \
    ssh://$IP:$PORT -t 4 -vV >> $LOG 2>&1
sleep 10
```

```
echo "--- FASE 2: Sesiones completas con sshpass (20 sesiones) ---" >> $LOG
for i in $(seq 1 20); do
```

```
U=${USERS[$((RANDOM % 8))]}
P=${PASSES[$((RANDOM % 8))]}
C=${COMMANDS[$((RANDOM % 5))]}
echo "[Sesion $i] Usuario: $U | Comando: $C" >> $LOG
sshpass -p "$P" ssh \
  -o StrictHostKeyChecking=no \
  -o ConnectTimeout=5 \
  -p $PORT root@$IP "$C" 2>/dev/null
sleep 3
done
```

```
echo "=== EXPERIMENTO FINALIZADO ===" >> $LOG
echo "Fecha fin: $(date)" >> $LOG
echo ""
echo "Experimento completado. Ver log en $LOG"
```

Diccionario de contraseñas utilizado (passwords.txt):

```
123456
password
admin
root
1234
qwerty
abc123
root123
pass
test
```

Anexo F — Workflow n8n completo

Captura del workflow completo de n8n con los doce nodos del pipeline SOAR-lite y sus conexiones. Los nodos verdes indican ejecución exitosa. La rama superior procesa todos los eventos hacia PostgreSQL y Elasticsearch. La rama inferior gestiona las alertas críticas con geolocalización y análisis de VirusTotal.



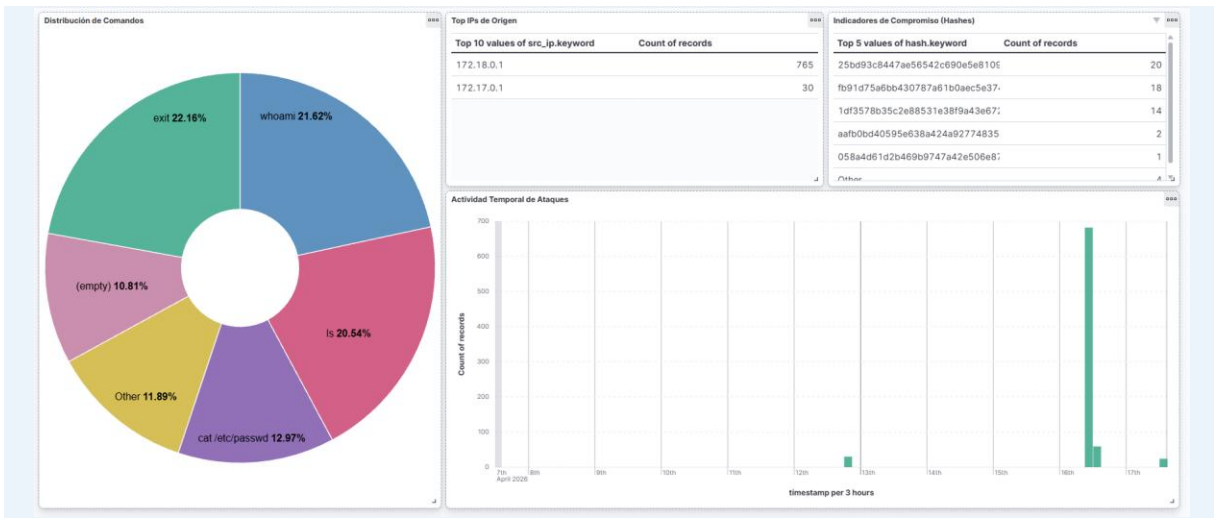
Descripción del flujo de datos entre nodos:

Nodo origen	Nodo destino	Datos transferidos
Trigger cada 60 seg	Leer cowrie_vps.json	Señal de activación temporal
Leer cowrie_vps.json	Parser NDJSON + Timestamp	Contenido binario del archivo de logs
Parser NDJSON + Timestamp	Filtrar eventos Cowrie	Array de eventos JSON nuevos
Filtrar eventos Cowrie	Guardar en PostgreSQL	Eventos con eventid que contiene 'cowrie'
Filtrar eventos Cowrie	Indexar en Elasticsearch	Mismos eventos (escritura paralela)
Filtrar eventos Cowrie	Filtrar eventos críticos	Mismos eventos (rama de alertas)
Filtrar eventos críticos	Geolocalización (ip-api)	Eventos login.success y file_download
Geolocalización (ip-api)	¿Descarga de archivo?	Evento original + datos geográficos
¿Descarga de archivo?	Consultar VirusTotal	Eventos file_download (rama true)
¿Descarga de archivo?	Alerta Login Exitoso	Eventos login.success (rama false)
Consultar VirusTotal	Alerta Descarga + VT	Evento + geolocalización + resultado VT

Anexo G — Dashboards de Kibana

El dashboard principal de Kibana reúne cuatro paneles de visualización analítica configurados sobre el índice de Elasticsearch que n8n alimenta en tiempo real. El período de visualización puede ajustarse libremente mediante el selector de fechas en la esquina superior derecha de la interfaz.

Fase 6 — Experimento Controlado: 40 Sesiones de Ataque Simulado (Hydra + sshpass)



Fase 7 y 8 — Tráfico Real de Internet: Ataques Capturados por el VPS (Amsterdam)



Descripción de cada panel del dashboard:

Panel	Tipo de visualización	Campo analizado	Valor analítico
Distribución de Comandos	Gráfico de dona	command.keyword	Muestra qué comandos ejecutan más frecuentemente los atacantes en la shell simulada
Top IPs de Origen	Tabla de datos	src_ip.keyword	Identifica las IPs más activas, permite detectar campañas de ataque coordinadas
Actividad Temporal de Ataques	Histograma de barras	timestamp (por 12 horas)	Revela patrones horarios de actividad maliciosa y picos de ataques
Indicadores de Compromiso	Tabla de datos	hash.keyword	Lista los hashes SHA-256 de archivos maliciosos descargados por los atacantes

Anexo H — Contenedores Docker activos (entorno local)

Captura del comando docker ps en PowerShell mostrando los cinco contenedores del sistema local en estado activo. Este estado corresponde al modo de operación normal con el pipeline completo funcionando.

```
PS C:\Users\Elizabeth> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b57dcb311625	cowrie/cowrie	"/cowrie/cowrie-env/..."	6 days ago	Up 10 hours	0.0.0.0:2222->2222/tcp, [::]:2222->2222/tcp	friendly_kalam
5b03b4cbb3cc	kibana:8.13.0	"/bin/tini -- /usr/l..."	9 days ago	Up 10 hours	0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp	kibana
288d657e5579	elasticsearch:8.13.0	"/bin/tini -- /usr/l..."	9 days ago	Up 10 hours	0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp	elasticsearch
664c5f78506e	n8nio/n8n	"tini -- /docker-ent..."	9 days ago	Up 10 hours	0.0.0.0:5678->5678/tcp, [::]:5678->5678/tcp	n8n
774b91ec6f8f	postgres	"docker-entrypoint.s..."	9 days ago	Up 10 hours	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp	db-honeypot

Detalle de los contenedores y sus mappings de puertos:

Nombre del contenedor	Imagen	Puerto host	Puerto contenedor	Servicio
friendly_kalam	cowrie/cowrie	2222	2222	Honeypot SSH local
db-honeypot	postgres	5433	5432	Base de datos PostgreSQL
n8n	n8nio/n8n	5678	5678	Motor de automatización
elasticsearch	elasticsearch	9200	9200	Motor de indexación
kibana	kibana	5601	5601	Visualización analítica

Anexo I — Honeypot VPS activo (servidor Amsterdam)

Captura del comando docker ps ejecutado dentro del servidor VPS de DigitalOcean (IP: 167.71.8.253, región Amsterdam AMS3) mostrando el contenedor Cowrie activo y expuesto a internet en el puerto 2222.

1 — Conexión al servidor VPS desde PowerShell:

```
ssh -i C:\Users\Elizabeth\.ssh\id_rsa_vps root@167.71.8.253
```

Nota: La autenticación se realiza mediante clave SSH RSA de 4096 bits. La clave privada se gestiona de forma local por los autores del proyecto.

2 — Actualizar el sistema e instalar Docker:

```
apt update && apt upgrade -y
curl -fsSL https://get.docker.com | sh
```

3 — Crear la carpeta de logs y asignar permisos:

```
mkdir -p /opt/honeypot/logs
chmod 777 /opt/honeypot/logs
```

4 — Desplegar el contenedor Cowrie:

```
docker run -d \
  --name cowrie \
  --restart always \
  -p 2222:2222 \
  -v /opt/honeypot/logs:/cowrie/cowrie-git/var/log/cowrie \
  cowrie/cowrie
```

5 — Verificar que Cowrie está activo:

```
docker ps
```

6 — Ver logs en tiempo real:

```
tail -f /opt/honeypot/logs/cowrie.json
```

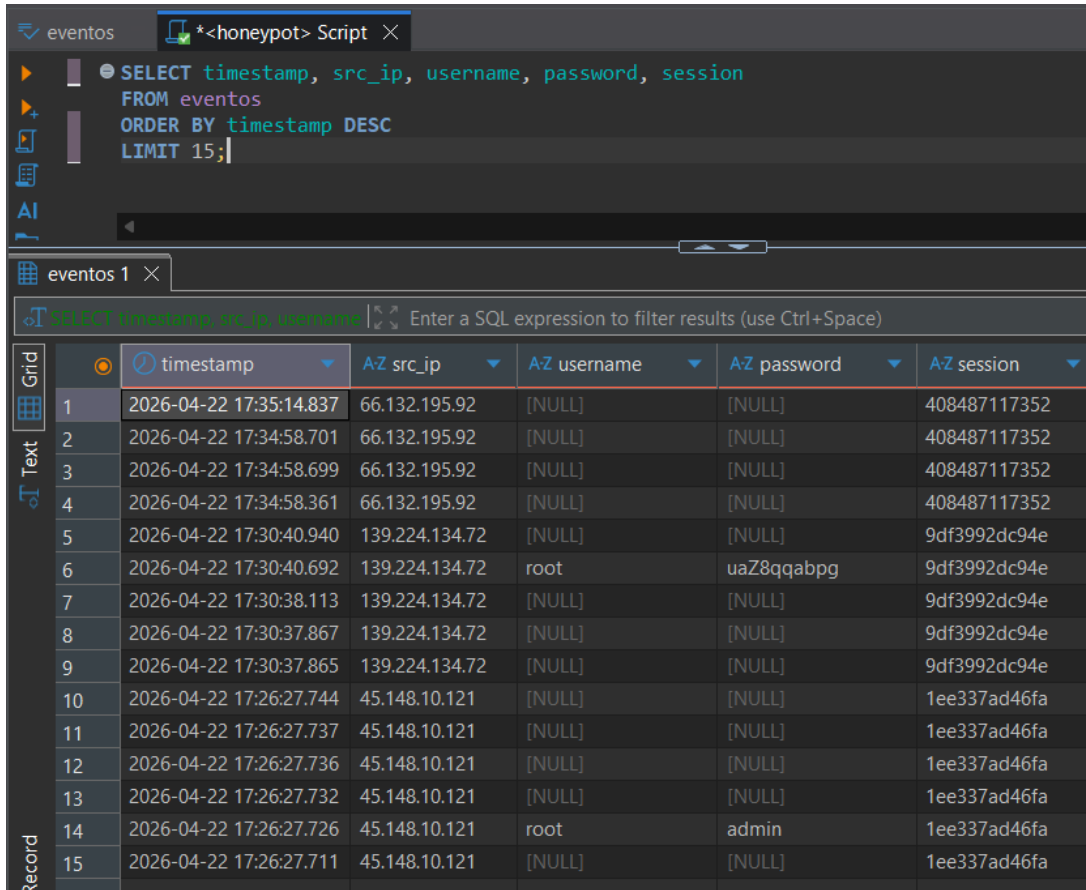
Especificaciones del servidor VPS:

Parámetro	Valor
Proveedor	DigitalOcean
Nombre del servidor	honeypot-oscorp
IP pública	167.71.8.253
Región	Amsterdam AMS3
Sistema operativo	Ubuntu 24.04 LTS
RAM	1 GB
Almacenamiento	25 GB SSD
Transferencia mensual	1000 GB incluidos
Costo mensual	USD 6
Puerto expuesto (Cowrie)	2222 (TCP)
Directorio de logs	/opt/honeypot/logs/cowrie.json

Anexo J — Resultados del experimento en DBeaver

Captura de la consulta SQL ejecutada en DBeaver sobre la base de datos PostgreSQL, mostrando los eventos más recientes registrados por el honeypot del VPS, correspondientes a atacantes reales de internet capturados de forma autónoma y el tráfico real capturado por el VPS.

Figura J.1 — Resultado de la consulta SQL en DBeaver. Se observan IPs reales de atacantes externos con sus credenciales utilizadas y sesiones únicas identificadas. Los campos NULL corresponden a eventos de conexión previos a la autenticación.



eventsos *-<honeypot> Script X

```
SELECT timestamp, src_ip, username, password, session
FROM eventos
ORDER BY timestamp DESC
LIMIT 15;
```

eventos 1 X

SELECT timestamp, src_ip, username Enter a SQL expression to filter results (use Ctrl+Space)

	timestamp	A-Z src_ip	A-Z username	A-Z password	A-Z session
1	2026-04-22 17:35:14.837	66.132.195.92	[NULL]	[NULL]	408487117352
2	2026-04-22 17:34:58.701	66.132.195.92	[NULL]	[NULL]	408487117352
3	2026-04-22 17:34:58.699	66.132.195.92	[NULL]	[NULL]	408487117352
4	2026-04-22 17:34:58.361	66.132.195.92	[NULL]	[NULL]	408487117352
5	2026-04-22 17:30:40.940	139.224.134.72	[NULL]	[NULL]	9df3992dc94e
6	2026-04-22 17:30:40.692	139.224.134.72	root	uaZ8qqabpg	9df3992dc94e
7	2026-04-22 17:30:38.113	139.224.134.72	[NULL]	[NULL]	9df3992dc94e
8	2026-04-22 17:30:37.867	139.224.134.72	[NULL]	[NULL]	9df3992dc94e
9	2026-04-22 17:30:37.865	139.224.134.72	[NULL]	[NULL]	9df3992dc94e
10	2026-04-22 17:26:27.744	45.148.10.121	[NULL]	[NULL]	1ee337ad46fa
11	2026-04-22 17:26:27.737	45.148.10.121	[NULL]	[NULL]	1ee337ad46fa
12	2026-04-22 17:26:27.736	45.148.10.121	[NULL]	[NULL]	1ee337ad46fa
13	2026-04-22 17:26:27.732	45.148.10.121	[NULL]	[NULL]	1ee337ad46fa
14	2026-04-22 17:26:27.726	45.148.10.121	root	admin	1ee337ad46fa
15	2026-04-22 17:26:27.711	45.148.10.121	[NULL]	[NULL]	1ee337ad46fa

Anexo K — Formato de alertas de Telegram

El sistema genera dos tipos de alertas automáticas enviadas por el bot de Telegram del proyecto OSCORP. A continuación se muestra el formato exacto de cada tipo de mensaje tal como es recibido por el operador en su dispositivo móvil.

K.1 — Alerta de Login Exitoso

Se genera cuando Cowrie registra un evento de tipo `cowrie.login.success`. Incluye geolocalización completa de la IP atacante obtenida automáticamente mediante `ip-api.com`.

```
📍 ALERTA DE INTRUSIÓN 📍
Tipo: LOGIN EXITOSO
IP Atacante: 47.251.178.17
🌐 Origen: United States, Minkler
ISP: Alibaba Cloud LLC
Usuario: root
Contraseña: P
```

```
Sesión: d7a9a1775046
Fecha: 21/4/2026
Hora: 10:56:14
❑ Detectado automáticamente por Honeypot OSCORP
This message was sent automatically with n8n
```

Nota: La IP 47.251.178.17 corresponde a un atacante real capturado por el honeypot del VPS durante las primeras 24 horas de operación. La geolocalización fue generada automáticamente por ip-api.com.

K.2 — Alerta de Descarga de Archivo con resultado VirusTotal

Se genera cuando Cowrie registra un evento de tipo `cowrie.session.file_download`. Incluye geolocalización de la IP atacante y el resultado del análisis automático del hash SHA-256 del archivo en VirusTotal.

```
🔊 ALERTA DE INTRUSIÓN 🔊
Tipo: DESCARGA DE ARCHIVO
IP Atacante: 201.190.175.63
🌐 Origen: Argentina, Mendoza
ISP: Arlink S.A.
Sesión: 3fc0bcf2e80b
Hash SHA-256: fb91d75a6bb430787a61b0aec5e374f5
                80030f2878e1613eab5ca6310f7bbb9a
🔍 Resultado VirusTotal:
Detectado por: 0/76 antivirus
✔ No detectado como malware
Fecha: 21/4/2026
Hora: 11:25:05
❑ Detectado automáticamente por Honeypot OSCORP
This message was sent automatically with n8n
```

Nota: El hash fb91d75a... corresponde al archivo estándar descargado desde example.com durante las pruebas. El resultado 0/76 antivirus es el esperado para este archivo de prueba. En ataques reales con malware genuino, el contador de detecciones sería significativamente mayor.